

AI 하는 사람을 위한 깃허브 초간단

AI 하는 사람을 위한 깃허브 초간단 — 목차

제품 포지셔닝

나두Ai의 『AI 하는 사람을 위한 깃허브 초간단』은 AI·코딩을 막 시작한 분이 GitHub.com에서 **계정 만들기 → 저장소 이해하기 → 클론 → 커밋 → 푸시 → Pull Request**까지, 화면을 따라가며 한 바퀴 돌릴 수 있게 돕는 초보용 전자책입니다. Git 심화, GitHub Actions, 대규모 모노레포는 다루지 않습니다. 브라우저와 github.com만 있으면 충분합니다. 나두Ai 영상·팁을 보고 “좋은데 내 PC·계정에서 어떻게 따라하지?”에서 멈춘 분도, 이 책으로 **같은 동작을 직접 재현**해 볼 수 있습니다.

부록 안내

본편 한 바퀴 뒤, @nadooai 인기 영상 주제별 **깃허브 실습**은 부록 「나두 인기 영상 실습」에서 이어갑니다. 1~9장 제목·순서는 그대로입니다.

무료 PDF: <https://tinalinkeom.vercel.app/ebook>

브랜드: 나두Ai

대상: AI·코딩을 막 시작한 한국어 사용자

전체 목차

장	제목	예상 페이지
1장	이 책으로 할 수 있는 것 / 안 다루는 것	4~5p
2장	GitHub 계정 만들기	4~5p
3장	저장소(Repository) 이해하기	4~5p
4장	클론(Clone) — 내 컴퓨터로 복사해 오기	4~5p
5장	커밋(Commit) — 변경을 기록하기	4~5p
6장	푸시(Push) — GitHub에 올리기	3~4p
7장	Pull Request(PR) — 변경을 제안하기	4~5p
8장	자주 막히는 장면 Q&A	3~4p
9장	다음에 연습할 것	2~3p
부록	나두 인기 영상 실습 (Lab 1~8)	별도
부록	깃허브 트렌드 실습 (Lab 1~8)	별도

장	제목	예상 페이지
고지	고지·법적 주의 (별도 장/부록)	별도
—	합계	약 32~41p (+부록·고지)

1장. 이 책으로 할 수 있는 것 / 안 다루는 것

학습 목표 - 이 책의 끝에서 스스로 할 수 있는 일 다섯 가지를 말할 수 있다 - “다루는 것”과 “안 다루는 것”을 구분할 수 있다 - Git과 GitHub의 차이를 한 문장으로 설명할 수 있다 - 따라하기에 필요한 준비물(브라우저, 인터넷, 이메일)을 확인한다

예상 페이지: 4~5p

2장. GitHub 계정 만들기

학습 목표 - github.com에서 새 계정을 만들 수 있다 - 이메일 인증과 기본 프로필 설정을 끝낼 수 있다 - 로그인 / 로그아웃을 스스로 할 수 있다 - 비밀번호·2단계 인증(2FA)의 기본 의미를 안다 (강제 설정은 선택)

예상 페이지: 4~5p

3장. 저장소(Repository) 이해하기

학습 목표 - 저장소(Repository)가 “프로젝트 폴더의 온라인 버전”임을 이해한다 - Public / Private의 차이를 말할 수 있다 - 빈 저장소를 github.com에서 새로 만들 수 있다 - README가 무엇인지, 왜 커 두면 좋은지 안다

예상 페이지: 4~5p

4장. 클론(Clone) — 내 컴퓨터로 복사해 오기

학습 목표 - 클론 = “원격 저장소를 내 컴퓨터로 통째로 복사해 오기”임을 이해한다 - github.com에서 Clone URL(HTTPS)을 복사할 수 있다 - 브라우저 또는 GitHub Desktop으로 클론하는 흐름을 따라 한다 - 클론이 끝난 뒤 “어디 폴더에 생겼는지” 확인할 수 있다

예상 페이지: 4~5p

5장. 커밋(Commit) — 변경을 기록하기

학습 목표 - 커밋 = “이 시점의 변경을 한 묶음으로 기록하기”임을 이해한다 - 파일을 조금 고친 뒤, 변경을 스테이징(선택)하고 커밋할 수 있다 - 커밋 메시지를 짧고 분명하게 쓰는 감을 익힌다 - “커밋만 하고 아직 GitHub에는 안 올린 상태”를 구분한다

예상 페이지: 4~5p

6장. 푸시(Push) — GitHub에 올리기

학습 목표 - 푸시 = “내 컴퓨터의 커밋을 GitHub 원격 저장소에 보내기”임을 이해한다 - 푸시 후 github.com에서 파일이 바뀐 것을 확인할 수 있다 - 푸시가 실패했을 때 먼저 볼 곳(로그인·권한·인터넷)을 안다 - 클론 → 커밋 → 푸시의 한 바퀴를 말로 설명할 수 있다

예상 페이지: 3~4p

7장. Pull Request(PR) — 변경을 제안하기

학습 목표 - PR = “내 변경을 본 저장소(또는 기본 브랜치)에 합쳐 달라고 제안하는 글”임을 이해한다 - 간단한 브랜치를 만들고, 그 위에서 커밋·푸시한 뒤 PR을 연다 - PR 제목·설명을 초보 수준으로 작성할 수 있다 - Merge(병합)의 의미를 “제안이 받아들여져 합쳐진 것”으로 이해한다

예상 페이지: 4~5p

8장. 자주 막히는 장면 Q&A

학습 목표 - 로그인·인증·권한 관련 흔한 막힘을 스스로 점검할 수 있다 - “어디에 올렸는지 모르겠다” 상황을 되짚는 순서를 안다 - 클론/푸시/PR에서 자주 보는 메시지를 두려워하지 않는다 - 이 책 범위를 넘는 문제일 때 “다음에 무엇을 찾아볼지” 방향을 안다

예상 페이지: 3~4p

9장. 다음에 연습할 것

학습 목표 - 혼자 반복 연습할 미니 미션 3~5개를 고른다 - “한 바퀴 더” 체크리스트로 복습한다 - 이 책 다음에 배우면 좋은 주제(브랜치 이름 규칙, Issue 등)를 가볍게 안다 - Actions·심화 Git은 일부러 나중에 두기로 한다

예상 페이지: 2~3p

범위 요약 (집필·배포 공통)

다름	안 다름
계정, 저장소, 클론, 커밋, 푸시, PR	GitHub Actions
github.com 화면 따라하기	Git 심화(리베이스, 체리픽 등)
HTTPS 클론·푸시 기본	대규모 모노레포
GitHub Desktop 가볍게 언급(선택)	VS Code 심화 설정 필수화

나두Ai · 화면 따라하기용

고지 · 법적 주의 (별도 장 / 부록)

위치: 본편 1~9장·실습 부록과 **별도**. 장 번호 1~9는 바뀌지 않습니다.

파일:

내용 요약: 저작권(무료 학습·무단 재판매 금지) · 제3자 상표(비제휴) · OSS·레포 · 나두Ai 영상과 PDF 관계 · 교육 면책 · 금지 안내 · 개인정보(최소) · **무료 제공(유료·환불 미적용)** · 배포자 표시

※ AI가 정리한 체크리스트이며 법률 의견이 아닙니다. 유료 재개 시 통신판매·세금은 사용자·전문가 확인이 필요합니다.

1장. 이 책으로 할 수 있는 것 / 안 다루는 것

이 장은 “버튼을 누르기 전에” 읽는 장입니다.

무엇을 끝내면 성공인지, 무엇을 기대하면 안 되는지부터 맞춰 두면, 뒤에서 막혀도 덜 당황합니다. > **나두 영상을 보고 오셨다면**

> 팁 영상에서 본 흐름이 “어렵다 / 내 화면에서는 어디를 누르지?”로 남았다면, 이 장부터 **범위만 맞춘 뒤** 2장에서 내 계정으로 직접 따라 하면 됩니다. 이론은 최소, **내 PC·GitHub에서 한 바퀴**가 목표입니다.

1. 이 책을 끝까지 따라가면 생기는 일

이 책을 화면 따라가기로 끝까지 하면, 아래 다섯 가지를 **스스로** 할 수 있게 됩니다.

1. **GitHub 계정을 만들고** 로그인한다
2. **저장소(Repository)** 를 하나 만든다
3. 그 저장소를 **클론(Clone)** 해서 내 컴퓨터(또는 Desktop 앱이 보는 폴더)로 가져온다
4. 파일을 조금 고친 뒤 **커밋(Commit)** 하고 **푸시(Push)** 해서 github.com에 반영한다
5. **Pull Request(PR)** 를 열어 “이 변경을 합쳐 주세요”라고 제안해 본다

한 줄로 말하면 이렇습니다.

계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

이 한 바퀴를, 초보 눈높이로 한 번 완주한다.

AI 도구로 코드를 받아 본 적이 있어도, “그걸 GitHub에 어떻게 올리지?”에서 멈추는 경우가 많습니다. 이 책은 그 빈칸만 메웁니다.

[화면: 책 전체 흐름을 한 장에 그린 간단 다이어그램 — 계정 → 저장소 → 클론 → 커밋 → 푸시 → PR]

2. 이 책이 다루는 것 (범위)

다루는 내용을 표로 정리합니다.

주제	이 책에서의 의미	어디서 연습하나
계정	github.com 가입·로그인	브라우저
저장소	온라인 프로젝트 폴더 만들기	github.com
클론	원격 저장소를 내쪽으로 복사해 오기	github.com + (선택) Desktop
커밋	변경을 “한 묶음”으로 기록하기	Desktop 또는 간단한 Git 도구
푸시	기록한 커밋을 GitHub로 보내기	위와 동일

주제	이 책에서의 의미	어디서 연습하나
PR	변경을 합쳐 달라고 제안하는 글	github.com

용어를 미리 한 줄씩만

- **GitHub**
코드를 올리고, 보고, 함께 고치는 **웹 서비스**입니다. 주소는 <https://github.com> 입니다.
- **Git**
파일 변경 이력을 기록하는 **도구(버전 관리)** 입니다. GitHub는 Git으로 만든 이력을 웹에서 보여 주는 쪽에 가깝습니다.
- **저장소(Repository, 줄여서 repo)**
프로젝트 파일이 들어 있는 **온라인 폴더**라고 생각하면 됩니다.
- **클론(Clone)**
그 온라인 폴더를 **내 컴퓨터로 통째로 복사해 오기**입니다.
- **커밋(Commit)**
“여기까지 이렇게 바꿨다”를 **한 번 기록**하는 것입니다. 아직 GitHub 사이트에 자동으로 올라가진 않습니다.
- **푸시(Push)**
내쪽에 쌓인 커밋을 **GitHub 저장소로 보내기**입니다.
- **Pull Request(PR)**
“이 변경을 기본 줄기에 합쳐 주세요”라고 **제안하는 요청 글**입니다.
(이름은 Pull이지만, 초보 단계에서는 “합쳐 달라는 제안서”로 기억해도 됩니다.)

여기서 외울 것

Git ≠ GitHub.

Git은 “기록하는 도구”, GitHub는 “그걸 올려 두고 보는 웹”에 가깝습니다.

이 책은 **GitHub 화면을 따라가는 것**이 중심이고, Git 명령어를 깊게 파지 않습니다.

[화면: github.com 첫 화면 — 로그인 전 또는 로그인 후 홈 중 하나]

3. 이 책이 안 다루는 것 (의도적으로 뺐)

초보가 한 번에 다 잡으려다 지치는 주제는 **일부러 빼** 두었습니다.

안 다룸	이유 (짧게)
GitHub Actions	자동 빌드·배포 파이프라인. 계정·커밋·PR 다음에 배워도 늦지 않음

안 다름	이유 (짧게)
Git 심화	리베이스, 체리픽, 복잡한 히스토리 수정 등
대규모 모노레포	회사·오픈소스 초대형 저장소. 권한·브랜치 규칙이 다름
VS Code 심화 설정	에디터 필수 설치·확장 프로그램 튜닝은 이 책 범위 밖
유료 팀·조직 관리	Organization 고급 권한, 결제·시트 관리

나중에 “그다음”이 궁금하면 9장에서 연습 미션만 가볍게 안내합니다.
 지금은 **한 바퀴 완주**가 목표입니다.

여기서 막히면

“Actions도 배워야 하나요?” → **아니요. 이 책에서는 안 합니다.**
 계정·클론·커밋·푸시·PR만 되면 1차 목표는 달성입니다.

4. 누가 읽으면 잘 맞나

아래 중 **하나라도** 해당하면 이 책 대상에 가깝습니다.

- AI 채팅·코드 생성 도구로 파일을 받아 봤는데, GitHub에 올리는 법이 어렵다
- “클론이 뭐예요?” “푸시가 뭐예요?”를 검색해 본 적이 있다
- 협업 전에, 혼자라도 **내 저장소에 올리고 PR까지** 연습해 보고 싶다
- 영어 화면이 부담되지만, **버튼 이름만 알면** 따라갈 수 있을 것 같다

반대로, 이미 매일 PR을 올리고 Actions까지 쓰는 분이라면 이 책은 **복습용·지인용** 수준입니다.
 새 고급 기법은 기대하지 않는 편이 좋습니다.

5. 따라하기 전에 준비할 것

복잡한 개발 환경은 필요 없습니다.

꼭 필요한 것

1. 인터넷이 되는 컴퓨터
2. 웹 브라우저 (Chrome, Edge, Safari, Firefox 등 — 평소 쓰는 것)
3. 이메일 주소 (GitHub 가입·인증에 사용)
4. 약 1~2시간 한 번에, 또는 장마다 나눠서 따라할 시간

있으면 편한 것 (필수는 아님)

- **GitHub Desktop** 앱
 클론·커밋·푸시를 버튼으로 하기 쉽습니다.
 이 책에서는 “있으면 이렇게” 정도로만 언급하고, **반드시 설치하라고** 강요하지 않습니다.

- 메모장이나 간단한 텍스트 에디터
README 파일 한 줄을 고칠 때 씁니다.
VS Code를 이미 쓰면 그걸 써도 되고, **없어도** 됩니다.

이 책이 가정하는 기본 경로

- 기본: 브라우저 + **github.com**
- 보조: GitHub Desktop (선택)
- 제외: 터미널 Git 명령어 마스터, VS Code Git 패널 심화

[화면: 준비물 체크리스트 — 인터넷 / 브라우저 / 이메일 / (선택) Desktop]

6. 어떻게 읽으면 좋은지 (화면 따라가기 약속)

이 책은 설명서처럼만 읽고 끝내지 말고, **화면을 같이 열고** 읽는 편을 권합니다.

읽는 순서 제안

1. 각 장의 **학습 목표**를 한 번 읽는다
2. 번호 매긴 **단계를** 그대로 따른다
3. [화면: ...] 표시가 나오면, 나중에 실제 캡처가 들어갈 자리입니다. 지금은 “그 부근을 보라”는 표시로 읽으면 됩니다
4. 「**여기서 막히면**」 상자가 나오면, 막혔을 때만 펼쳐 봐도 됩니다
5. 한 장이 끝나면, 장의 끝 **체크**로 “했다 / 안 했다”만 표시한다

실패해도 되는 연습

초보 GitHub는 “한 번에 완벽”보다 **한 번 끝까지**가 중요합니다.
잘못된 커밋 메시지, 어색한 PR 제목은 연습용 저장소에서 얼마든지 다시 하면 됩니다.

여기서 막히면

“내 코드가 망가질까 봐 무섭다” →

연습용으로 **빈 저장소**나 **README만 있는 저장소**를 새로 만드세요.

중요한 회사 저장소·남의 저장소에는 이 책 연습을 바로 적용하지 마세요.

7. GitHub 화면이 영어인 이유 (짧게)

github.com 기본 화면은 영어인 경우가 많습니다.

이 책에서는 버튼·메뉴를 **한글 설명 + 영어 원문**으로 같이 적습니다.

예:

- **Sign up** = 가입하기

- **New repository** = 새 저장소
- **Clone** = 복사해 오기
- **Commit** = 기록하기
- **Push** = 올리기
- **Pull request** = 합쳐 달라는 제안

단어를 전부 외울 필요는 없습니다.

그 **장에서 누르는 버튼 이름만** 기억하면 됩니다.

[화면: github.com 상단 메뉴 — 영어 라벨에 한글 화살표 메모가 붙은 예시(가짜 얼굴·실명 없이)]

8. 안전·매너 약속 (짧게, 과장 없이)

- **비밀번호**는 이 책에 적지 마세요. 누구에게도 보내지 마세요.
- **남의 저장소**에 함부로 푸시하거나, 허락 없이 큰 변경 PR을 보내지 마세요.
- 연습은 **본인 계정·본인이 만든 저장소**에서 하세요.
- 이 책은 **법률·세금·결제 대행 안내가 아닙니다**. 판매·입금 방법은 판매 페이지의 안내를 따릅니다.

나두Ai는 얼굴·실명을 내세우지 않는 **익명 브랜드**로, 이 책도 “따라하기 교재”에 집중합니다.

9. 이후 장에서 할 일 미리보기

장	한 줄 예고
2장	github.com에서 계정을 만든다
3장	저장소가 뭔지 보고, 하나를 만든다
4장	클론으로 내쪽에 복사해 온다
5장	파일을 고치고 커밋한다
6장	푸시로 GitHub에 반영한다
7장	PR을 열어 합쳐 달라고 제안한다
8장	자주 막히는 장면을 Q&A로 정리한다
9장	혼자 반복할 미니 미션을 고른다

지금 당장 2장으로 넘어가도 됩니다.

다만 “Actions까지 해야 하나?” “Git 명령어를 다 외워야 하나?”라는 불안이 있다면,
이 1장만으로도 **범위가 좁혀진 상태**가 됩니다. 그게 목적입니다.

10. 1장 체크리스트

아래를 소리 내어(또는 메모로) 답해 보세요.

- ☐ 이 책을 끝내면 **클론·커밋·푸시·PR**까지 한 바퀴 돌릴 수 있다고 말할 수 있다
- ☐ **안 다루는 것**(Actions, Git 심화, 모노레포)을 알고 있다
- ☐ Git과 GitHub의 차이를 **한 문장**으로 말해 볼 수 있다
- ☐ 준비물(브라우저, 이메일, 인터넷)을 확인했다
- ☐ 연습은 **내 저장소**에서 하겠다고 정했다

모두 체크되면 2장으로 갑니다.

여기서 막히면 (1장 전용)

Q. PDF가 아니라 웹 강의가 필요한가요?

A. 이 제품은 **전자책(PDF)** 따라하기입니다. 화면 캡처는 이후 장·개정에서 채워질 수 있고, 지금은 [화면: ...] 자리를 보고 github.com을 직접 열면 됩니다.

Q. 맥인가요, 윈도우인가요?

A. 둘 다 가능합니다. 브라우저로 github.com만 되면 2장까지는 차이가 거의 없습니다. Desktop 앱을 쓸 때는 각 OS 설치 화면만 다를 수 있습니다.

Q. 코딩을 전혀 못해도 되나요?

A. 됩니다. 이 책은 “코드를 잘 짜기”가 아니라 **GitHub에 올리고 제안하는 최소 동작**이 목표입니다. README에 한 줄 적는 정도면 연습에 충분합니다.

Q. 이미 계정이 있으면 1~2장을 건너뛰어도 되나요?

A. 계정만 있다면 **2장은 훑어보고**, 3장(저장소)부터 따라가도 됩니다. 다만 1장의 “범위”는 한 번 읽는 것을 권합니다.

1장을 마치며

이 장의 핵심만 다시 적습니다.

- 목표는 **계정 → 저장소 → 클론 → 커밋 → 푸시 → PR** 한 바퀴
- Actions·심화 Git·대형 저장소는 **안 다룸**
- 준비물은 **브라우저 + 이메일**이 중심
- 영어 버튼은 **그 장에서 누르는 것만** 기억
- 연습은 **내 저장소**에서

다음 장에서는 실제로 <https://github.com>에 들어가 **계정을 만듭니다**.

계정이 이미 있다면, 로그인만 확인한 뒤 3장으로 넘어가도 됩니다.

[화면: 2장으로 이어지는 안내 — github.com Sign up 버튼 부근] ## 따라 하기 체크 (1장)

- ☐ 영상에서 들은 “계정→저장소→클론→커밋→푸시→PR”이 **이 책 목표**와 같다고 확인했다

- ☐ **안 다루는 것**(Actions·심화 Git 등)을 기대하지 않기로 했다
- ☐ 준비물(브라우저·이메일)을 확인했다 → 다음은 **내 계정**으로 2장

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

2장. GitHub 계정 만들기

이 장에서는 브라우저만 열고, <https://github.com> 에서 **계정을 만들고 → 이메일을 인증하고 → 로그인·로그아웃**까지 스스로 해 봅니다.

VS Code나 Desktop 앱은 이 장에서 필요 없습니다.

화면 문구는 GitHub가 가끔 바꿉니다. 버튼 글자가 조금 달라도 **비슷한 문구면** 그 버튼을 누르면 됩니다. >
나두 영상을 보고 오셨다면

> 가입·로그인 화면을 영상으로만 봤다면, 여기서는 **본인 이메일로 계정을 만들고** 인증까지 끝내는 실습입니다. 버튼 문구가 조금 달라도 같은 흐름이면 OK.

학습 목표

- github.com에서 **새 계정을 만들** 수 있다
- **이메일 인증**과 **기본 프로필** 설정을 끝낼 수 있다
- **로그인 / 로그아웃**을 스스로 할 수 있다
- 비밀번호·**2단계 인증(2FA)**의 기본 의미를 안다 (강제 설정은 선택)

1. 시작 전에 준비 확인

1장에서 말한 준비물만 있으면 됩니다.

1. 인터넷이 되는 컴퓨터
2. 평소 쓰는 **웹 브라우저**
3. 본인이 열어 볼 수 있는 **이메일 주소**
4. 비밀번호를 **안전한 곳에** 적을 메모(종이·비밀번호 관리 앱 등). 이 책·채팅·메신저에는 적지 마세요

이미 계정이 있다면?

가입 단계(2~4절)는 건너뛰고 **5절 → 6절 → 7절**만 따라가도 됩니다.

비밀번호·2FA 의미는 8절을 한 번 읽어 두세요.

여기서 막히면

“예전에 가입한 것 같은데 기억이 안 난다” →

새 가입 전에 **Sign in**(로그인) → **Forgot password?**(비밀번호 찾기)로

이메일 재설정을 먼저 시도하세요. 같은 이메일로 계정이 있으면 새 가입이 헛갈릴 수 있습니다.

[화면: github.com 첫 화면 — Sign up / Sign in 버튼 부근]

2. github.com 열고 Sign up 누르기

단계

1. 주소창에 `https://github.com` 을 입력합니다. (`https://` 포함)
2. **Sign up**(가입하기) 비슷한 버튼을 클릭합니다.
3. 바로 가려면 `https://github.com/signup` 으로 열어도 됩니다.

[화면: Sign up 직후 — Create your free account / Sign up for GitHub 비슷한 제목]

이 책은 **개인 무료 계정(Free)** 기준입니다.

화면에 Pro·Team 안내가 보여도 지금은 **무료로 계속**하면 됩니다. 유료·조직 결제는 다루지 않습니다.

여기서 막히면

“회사 메일로 해야 하나요?” → **아니요**. 매일 확인하는 개인 이메일이면 충분합니다.

임시(일회용) 메일은 인증이 안 되거나 막힐 수 있으니 쓰지 마세요.

3. 사용자 이름 · 이메일 · 비밀번호 입력하기

가입 화면은 한 페이지에 다 나오거나, **Continue**로 한 칸씩 넘어갈 수 있습니다.

묻는 내용만 같으면 그대로 따라가면 됩니다.

3-1. 이메일 (Email)

본인이 **지금** 열어 볼 수 있는 주소를 입력하고, 오타를 한 번 더 확인합니다.

인증 메일이 이 주소로 옵니다.

3-2. 비밀번호 (Password)

GitHub는 대략 이런 규칙을 안내하는 경우가 많습니다. (문구는 바뀔 수 있습니다.)

- **15자 이상**, 또는
- **8자 이상**이면서 **숫자 + 소문자** 포함

다른 사이트와 **같은 비밀번호를 재사용하지 않는** 편이 안전합니다.

비밀번호를 이 책 · 캡처에 남기지 마세요.

여기서 막히면

“비밀번호가 거절된다” → 화면의 빨간 안내(길이·숫자·소문자)를 그대로 맞춥니다.

3-3. 사용자 이름 (Username)

Username은 주소의 일부가 됩니다. 예: `https://github.com/여기에-사용자이름`

흔한 규칙(안내 문구는 비슷합니다):

- 영문·숫자, **하이픈(-)** 정도만 허용
- **맨 앞·맨 뒤 하이픈**은 안 되는 경우가 많음

- 이미 쓰인 이름은 Unavailable(사용 불가)처럼 표시

실명·회사명을 꼭 넣을 필요는 없습니다.

이미 쓰인 이름이면 짧은 단어나 숫자를 붙이거나 다른 후보를 시도하세요.

여기서 막히면

“원하는 이름이 전부 사용 중” → 완벽할 필요 없습니다.

당장 가입 가능한 이름을 고르세요. 표시 이름(Name)은 나중에 프로필에서 다르게 둘 수 있습니다.

3-4. 국가/지역 · 약관 · Continue

Country/Region(국가/지역)을 물으면 본인 거주 국가를 선택합니다.

마케팅 메일 체크는 화면 문구를 읽고, 원하지 않으면 끄면 됩니다.

이용 약관·개인정보 안내는 가입 전 한 번 훑어 보세요. (이 책은 법률 대행이 아닙니다.)

3-5. 사람 확인(Captcha)

퍼즐·체크 상자가 나오면 안내에 따라 완료한 뒤

Create account / Continue 비슷한 버튼을 누릅니다.

[화면: 이메일·비밀번호·Username 입력란과 Continue — 실제 값은 가림]

여기서 막히면

“Captcha가 끝없이 실패한다” → 광고 차단 확장을 잠시 끄거나,

다른 브라우저로 <https://github.com/signup> 을 열어 보세요.

여러 번 실패했다면 잠시 쉬었다가 다시 시도합니다.

4. 이메일 인증하기 (Verify email)

인증이 끝나지 않으면 저장소 만들기 같은 **기본 작업이 막힐 수** 있습니다. 이 장에서 꼭 끝냅니다.

단계

1. 가입에 쓴 **이메일 편지함**을 엽니다.
2. GitHub가 보낸 메일에서 **인증 링크** 또는 **코드** 안내를 찾습니다.
제목에 verify / confirm / launch code처럼 **확인** 관련 단어가 자주 들어갑니다.
3. 링크를 누르거나 코드를 입력합니다.
4. github.com으로 돌아가며 로그인(또는 로그인 유도)되면 성공에 가깝습니다.

[화면: “이메일을 확인하세요” 안내 — 실제 이메일 주소 가림]

여기서 막히면

“인증 메일이 안 온다” → 위에서부터 확인하세요.

1. 스팸·프로모션함

2. 가입 주소 오타
3. 1~2분 대기 후 화면의 **Resend**(다시 보내기)
4. 임시 메일이면 **일반 이메일**로 다시 가입 검토
5. 링크는 **시간 만료**될 수 있음 → Resend로 새 메일

로그인은 됐는데 인증만 안 됐다면:

아바타 → **Settings** → **Emails** → **Resend verification email** 비슷한 버튼.

5. 첫 로그인 확인하기

가입·인증 직후에는 이미 로그인된 경우가 많습니다. 화면만 확인합니다.

로그인된 상태의 흔한 모습

- 오른쪽 위에 **프로필 아바타**
- Sign up 대신 알림·+·아바타 메뉴
- 홈/대시보드·피드가 보일 수 있음

[화면: 로그인 후 상단 바 — 아바타 메뉴(실명·얼굴 없이)]

로그인이 풀렸을 때 (Sign in)

1. <https://github.com> 또는 <https://github.com/login>
2. **Sign in** 클릭
3. **Username** 또는 **이메일** + **비밀번호** 입력 후 Sign in

비밀번호를 잊었다면 **Forgot password?** 로 재설정 메일을 받습니다.

여기서 막히면

“I already have an account / 이미 계정이 있다” →

새 가입이 아니라 **Sign in**으로 들어가세요.

이 책은 **이메일+비밀번호** 경로를 기본으로 설명합니다.

6. 기본 프로필 살짝 설정하기 (얼굴 사진 필수 아님)

프로필은 “명함” 정도입니다. **얼굴·실명은 필수**가 아닙니다. 익명으로 연습해도 됩니다.

단계 (최소)

1. 오른쪽 위 **아바타** 클릭
2. **Your profile** 또는 **Settings** 열기
3. 아래만 확인

항목	권장	메모
Avatar	선택	기본 아이콘 그대로 OK. 얼굴 사진 강요 없음
Name	선택	Username과 달라도 됨. 실명 불필요
Bio	선택	비우거나 “학습 중” 한 줄
Email	확인	인증된 메일이 Primary인지 정도

아바타를 바꾼다면 Settings → **Profile**에서
로고·일러스트·이니셜 등 얼굴이 아닌 그림도 충분합니다.
 Save / Update profile 비슷한 버튼으로 저장합니다.

[화면: Settings → Profile — 아바타·Name 칸(개인정보 가림)]

이 장에서는 “문이 열렸는지”만 보면 됩니다. 공개 활동은 나중에 다듬어도 됩니다.

7. 로그아웃했다가 다시 로그인하기 (연습)

로그아웃 (Sign out)

1. 아바타 클릭
2. **Sign out** 클릭
3. Sign in / Sign up이 다시 보이면 로그아웃된 것

[화면: 아바타 드롭다운 — Sign out 강조(실명 없이)]

다시 로그인 (Sign in)

1. **Sign in** → Username(또는 이메일) + 비밀번호
2. 아바타가 다시 보이면 성공

공용 PC에서는 실습 후 반드시 **Sign out** 하세요.

8. 비밀번호와 2단계 인증(2FA) — 의미만 알고 가기

이 절은 강제 설정 실습이 아닙니다.

비밀번호

본인만 github.com에 들어오게 하는 열쇠입니다.
 채팅·이메일·스크린샷에 남기지 마세요.

2FA (Two-factor authentication)

비밀번호 외에 **두 번째 확인**(휴대폰 앱 코드, 보안 키 등)을 요구하는 잠금입니다.
비밀번호만 새어도 계정을 지키기 유리합니다.
가입 직후나 Settings에서 2FA를 켜 보라는 안내가 나올 수 있습니다.

이 책에서의 선택

- **권장(나중)**: 한 바퀴 익숙해진 뒤 Settings → Password and authentication 에서 켜다.
- **지금**: 마법사가 뜨면 GitHub 공식 단계를 따르거나,
Skip for now / 나중에가 허용되면 이 책을 먼저 완주해도 된다.
- 인증 앱·복구 코드 보관까지 **깊게 강제하지 않습니다**.
복구 코드가 보이면 **안전한 곳에만** 두고 책에 적지 마세요.

여기서 막히면

“2FA 없으면 이 책을 못 하나요?” → **아니요**.

목표는 계정·이메일 인증·로그인입니다.

다만 GitHub 정책으로 **일부 기능에 2FA가 필요할** 수 있습니다. 그때는 공식 화면 안내를 우선하세요.

[화면: (선택) 2FA 안내 배너 또는 Settings 보안 메뉴 — 복구 코드·QR 가림]

9. 이 장에서 쓰지 않는 것

안 함	이유
VS Code	계정만 있으면 됨
GitHub Desktop	4장 이후 선택
터미널 git	범위 밖
유료·Organization	이 책 범위 밖
얼굴·실명 공개	익명 연습 가능

기본 경로: 브라우저 + <https://github.com>

10. 2장 체크리스트

- ☐ <https://github.com> 에서 **계정을 만들었거나** 기존 계정으로 **로그인**할 수 있다
- ☐ **이메일 인증**을 끝냈다 (또는 이미 인증됨)
- ☐ 오른쪽 위 **아바타 메뉴**를 열 수 있다
- ☐ 프로필에서 **아바타는 선택**, 얼굴 사진이 필수가 아님을 안다
- ☐ **Sign out** 후 **Sign in**을 한 번 연습했다
- ☐ 비밀번호를 책에 적지 않았고, **2FA는 의미만** 안다 (설정은 선택)

모두 체크되면 3장으로 갑니다.

여기서 막히면 (2장 전용 Q&A)

Q. 가입 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다. **Email / Password / Username / Verify** 흐름만 같으면 같은 단계입니다. 비슷한 의미의 버튼을 누르세요.

Q. Username을 나중에 바꿀 수 있나요?

A. 가능한 경우가 많지만 기존 링크가 깨질 수 있습니다. 초보 때는 **신중히 고른 뒤 당분간 유지**하는 편이 덜 헛갈립니다.

Q. 다른 로그인 방식으로 가입 버튼이 보여요.

A. 이 책은 **이메일+비밀번호**를 기본으로 설명합니다. 다른 방식은 공식 안내를 따르되, 책의 버튼 위치와 달라질 수 있습니다.

Q. 인증 후 “Create a repository” 안내가 보여요.

A. 잘 된 것입니다. **지금은 저장소를 급히 만들지 말고**, 체크리스트를 끝낸 뒤 **3장**에서 만듭니다.

Q. 비밀번호를 다른 사람에게 물어봐도 되나요?

A. 안 됩니다. 비밀번호·복구 코드·인증 링크는 본인만 다룹니다.

2장을 마치며

1. 주소는 **https://github.com** (가입은 signup)
2. **이메일 · 비밀번호 · Username** → 사람 확인 → 계정 생성
3. **이메일 인증**까지가 한 세트
4. **프로필**은 최소, 얼굴 사진 필수 없음
5. **Sign out / Sign in** 한 번 연습
6. **2FA**는 권장·선택 — 이 책에서 강제 심화 실습 없음

다음 **3장**에서는 **저장소(Repository)**가 무엇인지 보고,
github.com에서 **연습용 저장소 하나**를 만듭니다.
Public / Private, README 정도만 알면 충분합니다.

[화면: 3장 예고 — New repository 버튼 부근(아직 누르지 말 것)] ## 따라 하기 체크 (2장)

- ☐ 영상에서 본 Sign up을 **내 이메일·Username**으로 완료했다
 - ☐ **이메일 인증**까지 끝냈다
 - ☐ Sign out → Sign in 한 번 해 봤다
 - ☐ 다음: 영상 속 “새 저장소”를 **내 계정**에서 3장으로 재현
-

3장. 저장소(Repository) 이해하기

이 장에서는 github.com에 로그인된 상태에서,
저장소(Repository)가 무엇인지 보고, **연습용 저장소 하나**를 직접 만듭니다.

클론(Clone)·커밋·푸시는 **아직 하지 않습니다**.
이 장은 “내 저장소가 github.com에 **보인다**”에서 끝납니다.
VS Code나 Desktop 앱도 **이 장에서 필요 없습니다**.

화면 문구는 GitHub가 가끔 바꿉니다. 버튼 글자가 조금 달라도 **비슷한 문구면** 그 버튼을 누르면 됩니다. >
나두 영상을 보고 오셨다면
> “Repository 만든다”는 말만 기억한다면, 이 장에서 **내 계정에 연습용 저장소 하나**를 실제로 만들어 두면 됩니다. 클론은 아직 X.

학습 목표

- 저장소(Repository)가 “**프로젝트 폴더의 온라인 버전**”임을 이해한다
- **Public / Private**의 차이를 말할 수 있다
- **빈 저장소**를 github.com에서 새로 만들 수 있다
- **README**가 무엇인지, 왜 커 두면 좋은지 안다

1. 저장소란? (비유만 먼저)

저장소는 영어로는 **Repository**, 줄여서 **repo(레포)**라고도 부릅니다.
한 줄로 기억하면 됩니다.

저장소 = 프로젝트 폴더의 온라인 버전

컴퓨터에 내문서/연습프로젝트 같은 폴더가 있듯,
GitHub에는 **같은 느낌의 폴더가 웹에 하나** 있다고 생각하면 됩니다.
그 안에 README, 코드 파일, 나중에 올릴 변경 이력이 자리합니다.

내 컴퓨터	GitHub
폴더	저장소(Repository)
폴더 이름	저장소 이름
폴더 안 파일	저장소 안 파일
(아직) 로컬만	인터넷에서 보고·공유할 수 있음

1장에서 말한 한 바퀴 중, 지금은 「저장소」 단계입니다.

계정 → **저장소** → 클론 → 커밋 → 푸시 → PR

[화면: 저장소 = 온라인 폴더 비유 다이어그램 — PC 폴더와 github.com 저장소가 나란히]

여기서 막히면

“리포지토리·레포·repository 단어가 다 나와요” →

같은 말입니다. 이 책에서는 **저장소**를 기본으로 쓰고, 필요할 때만 Repository / repo를 병기합니다.

2. Public과 Private — 누구까지 보이나

저장소를 만들 때 **공개 범위**를 고릅니다.

초보가 겁먹을 주제는 아닙니다. “누가 볼 수 있나”만 구분하면 됩니다.

종류	의미 (초보용)	연습할 때
Public	인터넷에 계정이 있는 사람이 내용을 볼 수 있는 편	나중에 공개 연습·포트폴리오용
Private	본인(과 초대한 사람) 위주로 보는 편	연습용으로 추천 (확신이 없으면)

이 책에서의 권장

- **연습용 첫 저장소**는 확신이 없으면 **Private** 을 고르세요.
- 비밀번호·개인키·실제 고객 데이터는 **어떤 저장소에도 올리지 마세요.**
Public이든 Private이든, 민감한 내용은 저장소에 두지 않는 것이 안전합니다.
- “Private이면 무조건 안전하다”까지는 이 책에서 약속하지 않습니다.
다만 **연습 파일을 남에게 굳이 보여줄 필요가 없다면 Private이 덜 신경 쓰입니다.**

유료·팀·조직(Organization)의 복잡한 권한은 **다루지 않습니다.**

지금은 **본인 계정에서 Public / Private 중 하나 고르기만** 합니다.

[화면: New repository 화면 — Public / Private 라디오(또는 비슷한 선택) 부근]

여기서 막히면

“나중에 Public으로 바꿀 수 있나요?” →

가능한 경우가 많습니다. 설정에서 바꿀 수 있는 항목이 있습니다.

다만 **한 번 Public에 올린 내용**은 이미 본 사람·복사본이 있을 수 있으니,
처음부터 올리면 안 되는 내용은 올리지 마세요.

이름 바꾸기·삭제 등은 이 장 깊이로 다루지 않습니다. **나중에** 필요할 때 공식 도움말을 보면 됩니다.

3. 새 저장소 만들기 — New repository

로그인한 뒤, github.com에서 저장소를 **새로** 만듭니다.

단계

1. 브라우저에서 <https://github.com> 을 엽니다.
2. **로그인된 상태**인지 확인합니다. (오른쪽 위에 아바타가 보이면 OK)
3. 오른쪽 위 + (플러스) 메뉴를 클릭합니다.
4. **New repository**(새 저장소) 비슷한 항목을 클릭합니다.

바로 가려면 주소창에 <https://github.com/new> 을 입력해도 됩니다.

[화면: 상단 + 메뉴 — New repository 강조(설명·얼굴 없이)]

여기서 막히면

“+ 버튼이 안 보여요” →

1. 정말 로그인했는지(Sign in)
2. 창 너비가 좁아 메뉴가 접혔는지
3. <https://github.com/new> 로 직접 이동

이메일 인증이 안 끝났다면 저장소 생성이 막힐 수 있습니다.

2장 **Verify email** 을 먼저 끝내 주세요.

4. 저장소 이름 정하기 (이름 규칙만 간단히)

Repository name(저장소 이름) 칸에 이름을 입력합니다.

연습용으로 쓸 이름 예시

아래 중 하나처럼 **연습임이 드러나는 이름**이면 충분합니다.

- hello-github
- my-first-repo
- practice-readme

완벽한 이름일 필요 없습니다. **지금 만들 수 있는 이름**이면 됩니다.

흔한 규칙 (안내 문구는 바뀔 수 있음)

- 영문·숫자, **하이픈(-)**, 밑줄(_) 정도를 쓰는 편이 안전합니다
- **공백**은 피하는 것이 좋습니다 (하이픈으로 이어 쓰기)
- 같은 계정 안에 **이미 있는 이름**은 다시 쓸 수 없습니다
- 대소문자를 섞어도 되지만, 초보는 **소문자 + 하이픈**만 써도 충분합니다

Description(설명)은 비워 두거나 “연습용” 한 줄만 적어도 됩니다. 필수는 아닙니다.

Owner(소유자)는 **본인 Username**이 선택된 상태면 됩니다.
Organization(조직) 목록이 보여도, 이 책 연습은 **개인 계정**을 고르세요.

[화면: Repository name 입력란 — hello-github 예시, Description은 비움]

여기서 막히면

“Name is not available / 이미 있다” →

같은 계정에 그 이름이 있는 것입니다.

hello-github-2 처럼 숫자나 단어를 조금 바꾸세요.

(다른 사람 계정에 같은 이름이 있는 것과는 별개입니다. **내 계정 안에서만** 중복을 보면 됩니다.)

5. Public / Private 고르기 + README 켜기

이름 아래(또는 근처)에 공개 범위와 **초기화 옵션**이 있습니다.

5-1. Public 또는 Private

- 연습용·확신이 없으면 → **Private**
- “공개해도 되는 연습”이라고 스스로 정했으면 → **Public** 도 가능

둘 중 **하나만** 고르면 됩니다. 이 장의 목표는 “만들기”입니다.

5-2. Add a README — 켜 두기 (권장)

Add a README 비슷한 체크 상자가 있으면 **켜** 둡니다(ON).

README가 뭔가요?

- 저장소를 열었을 때 **맨 먼저 보이는 안내 파일**입니다.
- 보통 파일 이름은 README.md 입니다.
- “이 폴더(저장소)가 무엇인지”를 **사람에게 읽어 주는 표지**에 가깝습니다.

왜 지금 켜 두나요?

1. 완전히 빈 저장소보다, **파일이 하나라도 있는 화면**이 초보에게 덜 막막합니다.
2. 다음 장(클론)에서 “뭔가 복사해 올 내용”이 생깁니다.
3. 5장에서 README 한 줄을 고치며 **커밋 연습**하기 좋습니다.

이 책에서는 **Add a README = ON** 을 기본 경로로 둡니다.

5-3. 지금은 건드리지 않아도 되는 것

화면에 더 많은 옵션이 보일 수 있습니다. 초보는 **무시해도** 됩니다.

옵션 (비슷한 문구)	이 장에서
Add .gitignore	끄거나 기본값 유지 — 안 다뤄도 됨
Choose a license	None / 비움 — 안 다뤄도 됨
Add README	ON 권장

[화면: Public/Private 선택 + Add a README 체크(ON) — .gitignore·license는 흐리게]

여기서 막히면

“README를 안 켜어요” →

저장소는 만들어졌을 수 있습니다.

완전히 빈 화면이면 “create a README” / “Add a README” 비슷한 안내가 보이기도 합니다.

그 링크로 README를 추가하거나, **새 연습 저장소를 README ON으로 다시** 만들어도 됩니다.

(이름만 겹치지 않게 하세요.)

6. Create repository 누르기

입력이 끝났으면 화면 아래(또는 우측)의

Create repository(저장소 만들기) 비슷한 버튼을 누릅니다.

몇 초 안에 **새 저장소 페이지**로 이동하면 성공입니다.

주소 형태는 대략 이렇게 보입니다.

`https://github.com/내Username/저장소이름`

예: Username이 nadu-learner 이고 저장소가 hello-github 이면

`https://github.com/nadu-learner/hello-github` 비슷한 주소입니다.

(예시는 가짜입니다. **본인 Username·본인이 정한 이름**을 확인하세요.)

[화면: Create repository 버튼 — 클릭 직전]

여기서 막히면

“버튼이 비활성(회색)이다” →

저장소 이름이 비었거나, 규칙에 안 맞거나, 중복인 경우가 많습니다.

이름 칸 아래 **빨간 안내 문구**를 읽고 고칩니다.

“404 / 권한” 비슷한 화면이면 → 로그인·이메일 인증·주소 오타를 점검하세요.

7. 만들어진 화면 둘러보기 (README가 있는 저장소)

Create 직후, **Code** 탭이 열린 저장소 홈이 보입니다.

지금은 **구경만** 합니다. 클론 버튼은 눌러도 되고, 복사만 해 두고 다음 장으로 미뤄도 됩니다.

7-1. 상단에서 확인할 것

1. **저장소 이름**이 제목처럼 보이는지
2. **Public** 또는 **Private** 표시가 의도한 것과 같은지
3. 탭 중 **Code** 가 선택된 느낌이 드는지

[화면: 새 저장소 홈 — 이름·Public/Private 배지·Code 탭(개인정보 가림)]

7-2. README 미리보기

페이지 본문에 **README.md** 내용이 렌더링되어 보입니다.
제목이 저장소 이름으로 자동 들어가 있는 경우가 많습니다.

- 위쪽(또는 파일 목록)에 README.md 파일이 보입니다.
- 아래쪽에 **미리보기(읽기 화면)** 가 보입니다.

지금 내용을 고치지 않아도 됩니다.

“표지가 생겼구나”만 확인하면 이 장 목표는 충분합니다.

[화면: README.md 미리보기 — 저장소 이름 제목만 보이는 상태]

7-3. Code 버튼 (다음 장 예고만)

초록색(또는 강조색) **Code** 버튼을 누르면
HTTPS 주소 복사·Desktop으로 열기 같은 메뉴가 나옵니다.

이 장에서는 클론하지 않아도 됩니다.

URL을 미리 복사해 뒀도 되지만, **실제 클론 실습은 4장에서** 합니다.

지금은 Code 메뉴가 **어디에 있는지**만 눈으로 확인하세요.

[화면: Code 버튼 드롭다운 — HTTPS / Clone 부근(아직 클론하지 말 것)]

7-4. 파일 목록이 거의 비어 보여도 정상

README만 켜 직후면 파일이 **한두 개**인 것이 정상입니다.

커밋 한 줄이 이미 만들어져 있을 수 있습니다.

(“Initial commit” 비슷한 메시지 — **지금은 의미만** “첫 기록” 정도로 알면 됩니다. 커밋 깊게 파는 것은 5장.)

8. 이 장에서 하지 않는 것

안 함	언제 / 왜
클론(Clone)	4장에서 함
커밋·푸시	5~6장

안 함	언제 / 왜
PR	7장
저장소 이름 변경·삭제	깊이 안 다름. 필요하면 나중에 설정·공식 도움말
Actions	이 책 범위 밖
VS Code 필수	이 장 불필요
.gitignore·라이선스 고르기	초보 필수 아님

기본 경로: 브라우저 + <https://github.com> 에서 저장소가 보이기

9. 3장 체크리스트

- ☐ 저장소를 “프로젝트 폴더의 온라인 버전” 이라고 말할 수 있다
- ☐ **Public** 과 **Private** 차이를 한 문장으로 말할 수 있다
- ☐ 확신이 없으면 연습용은 **Private** 을 골라도 된다고 안다
- ☐ <https://github.com/new> (또는 + → New repository)로 저장소를 만들었다
- ☐ **Add a README** 를 켜 두었고, 저장소 페이지에서 README 미리보기를 봤다
- ☐ **Code** 버튼 위치를 확인했고, 클론은 4장으로 미뤘다

모두 체크되면 4장으로 갑니다.

여기서 막히면 (3장 전용 Q&A)

Q. 가입 직후 “Create a repository” 안내가 보였는데, 이미 만들어 버렸어요.

A. 괜찮습니다. README가 켜져 있고 본인 저장소면 그대로 써도 됩니다.

이름이 연습용으로 어색하면 새 저장소를 **README ON**으로 하나 더 만들어도 됩니다.

Q. Private으로 만들었는데 친구에게 보여 주고 싶어요.

A. 초대(Collaborator) 기능이 있습니다. 이 책 초보 범위에서는 깊이 다루지 않습니다.

지금은 본인만 보는 연습으로 두고, 필요하면 나중에 설정에서 초대하면 됩니다.

Q. README.md를 지워도 되나요?

A. 연습 중이라면 가능하지만, 이 책 다음 장에서 README를 활용합니다.

지금은 남겨 두는 것을 권합니다.

Q. 저장소가 여러 개 생겨 헷갈려요.

A. 아바타 → **Your repositories** (또는 프로필의 Repositories 탭)에서 목록을 볼 수 있습니다.

연습에 쓸 하나만 골라 주소를 북마크해 두면 덜 헷갈립니다.

Q. “Quick setup” / 빈 저장소 안내만 보여요.

A. README를 안 켜 상태로 만든 경우가 많습니다.

화면에 README 만들기 안내가 있으면 따라가거나, README ON으로 **새 저장소**를 만들면 이 책 경로와 맞춰집니다.

Q. 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다. **이름 입력 → Public/Private → README → Create** 흐름만 같으면 같은 단계입니다. **비슷한 문구**의 버튼을 누르세요.

3장을 마치며

1. 저장소 = **프로젝트 폴더의 온라인 버전**
2. **Public** = 비교적 공개, **Private** = 본인 위주 — 연습은 확신이 없으면 **Private**
3. + → **New repository** (또는 /new)로 만든다
4. 이름은 간단·연습용으로, **Add a README = ON**
5. Create 후 **Code** 탭·**README 미리보기**만 확인하면 이 장 완주
6. **클론**은 다음 장 — 지금은 github.com에서 **보이기**만

다음 **4장**에서는 이 저장소를 **클론(Clone)** 해서
내 컴퓨터(또는 Desktop 앱이 보는 폴더)로 **통째로 복사해 옵니다**.
Code 버튼과 HTTPS 주소를 실제로 쓰게 됩니다.

[화면: 4장 예고 — Code 버튼과 HTTPS Clone URL 부근] ## 따라 하기 체크 (3장)

- ☐ 영상에서 본 New repository를 **내 계정**에서 만들었다
 - ☐ Public/Private·**README ON**을 확인했다
 - ☐ Code 탭·README가 **내 저장소 URL**에 보인다
 - ☐ 다음: 그 저장소를 **내 PC로 클론**(4장)
-

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

4장. 클론(Clone) — 내 컴퓨터로 복사해 오기

이 장에서는 3장에서 만든 **저장소(README 포함)** 를 **클론(Clone)** 해서 **내 컴퓨터의 폴더**로 통째로 가져옵니다.

기본 경로는 **GitHub Desktop** 입니다.
브라우저에서 **HTTPS 주소**를 복사하는 방법도 함께 봅니다.
커밋·푸시는 **아직 하지 않습니다.** (5~6장)

화면 문구는 GitHub·Desktop이 가끔 바뀝니다.
버튼 글자가 조금 달라도 **비슷한 문구**면 그 버튼을 누르면 됩니다. > **나두 영상을 보고 오셨다면**
> 클론·Desktop 장면을 영상으로만 봤다면, 여기서는 **3장에서 만든 내 저장소**를 내 컴퓨터 폴더로 한 번 복사해 오는 실습입니다. 커밋·푸시는 다음 장.

학습 목표

- 클론 = “원격 저장소를 내 컴퓨터로 통째로 복사해 오기” 임을 이해한다
- github.com에서 **Clone URL(HTTPS)** 을 복사할 수 있다
- **GitHub Desktop**(또는 안내된 방법)으로 클론하는 흐름을 따라 한다
- 클론이 끝난 뒤 **어디 폴더에 생겼는지** 확인할 수 있다

1. 클론이란? (한 줄 + 비유)

클론은 영어로는 **Clone** 입니다.

한 줄로 기억하면 됩니다.

클론 = GitHub에 있는 저장소를, 내 컴퓨터로 통째로 복사해 오기

3장에서 만든 것은 **온라인 폴더**입니다.

클론은 그 온라인 폴더의 **사본**을 내 **PC**에 하나 더 만드는 것에 가깝습니다.

GitHub (원격)	클론 후 (내 컴퓨터)
github.com/내Username/저장소이름	예: Documents/GitHub/저장소이름 같은 로컬 폴더
README.md가 웹에서 보임	같은 README.md가 파일 탐색기 에도 있음
나중에 커밋·푸시할 준비	Git 이력이 함께 복사됨 (ZIP만 받는 것과 다름)

1장에서 말한 한 바퀴 중, 지금은 「**클론**」 단계입니다.

계정 → 저장소 → **클론** → 커밋 → 푸시 → PR

[화면: 클론 비유 다이어그램 — github.com 저장소 → 화살표 → PC 폴더]

여기서 막히면

“다운로드랑 뭐가 달라요?” →

브라우저 **Download ZIP** 은 파일만 압축해 받는 쪽에 가깝고,

클론 은 “나중에 커밋·푸시할 수 있는 연결”까지 함께 가져오는 편입니다.

자세한 차이는 바로 아래 박스만 읽으면 됩니다.

2. 잠깐: ZIP 받기 ≠ 클론

github.com의 **Code** 메뉴에 **Download ZIP** 이 보일 수 있습니다.

	Download ZIP	클론(Clone)
무엇	파일을 압축본으로 받기	저장소를 Git 과 함께 복사
이후 커밋·푸시	이 책 경로로는 준비되지 않은 상태에 가깝음	Desktop 등으로 이어서 하기 좋음
이책 권장장	구경만 / 비상용	이 장의 기본 경로

여기서 외울 것

연습 한 바퀴(클론 → 커밋 → 푸시)를 돌리려면 **클론**을 쓰세요.

ZIP만 받아 두면, 5~6장에서 “어디에 커밋하지?”가 막힐 수 있습니다.

이 장에서는 **Download ZIP**을 누르지 않아도 됩니다.

3. 시작 전에 확인

다음이 준비되어 있으면 됩니다.

1. **GitHub** 계정에 로그인할 수 있다 (2장)
2. **README**가 있는 **저장소**가 하나 있다 (3장)
3. 인터넷이 되는 컴퓨터
4. (기본 경로) **GitHub Desktop** 을 설치할 수 있는 권한
 - 회사 PC 등에서 설치가 막히면 **7절(터미널 안내)** 또는 관리자 문의를 보세요. 이 책은 Desktop을

권장합니다.

저장소 주소를 북마크해 두면 편합니다.

형태: `https://github.com/내Username/저장소이름`

[화면: 3장에서 만든 저장소 홈 — README가 보이는 Code 탭]

여기서 막히면

“저장소가 없어요 / README가 없어요” →

3장으로 돌아가 **Add a README = ON** 으로 연습용 저장소를 하나 만드세요.

완전히 빈 저장소도 클론은 되지만, 이 책은 **README가 있는 상태**를 기준으로 설명합니다.

4. Code 버튼 → HTTPS 주소 복사하기

클론에 쓰는 주소를 먼저 확인합니다.

Desktop만 쓸 때도, **HTTPS가 어디에 있는지** 알아 두면 막힐 때 도움이 됩니다.

단계

1. 브라우저에서 본인 저장소 페이지를 엽니다.
(`https://github.com/내Username/저장소이름`)
2. 초록색(또는 강조색) **Code** 버튼을 클릭합니다.
3. 탭·오피션 중 **HTTPS** 가 선택된 상태인지 확인합니다.
(SSH 가 보이면, 이 장에서는 **HTTPS** 를 고르세요. SSH 키 설정은 **나중에·선택**입니다.)
4. 주소 옆 **Copy**(복사) 아이콘을 누릅니다.
주소는 대략 이런 형태입니다.
`https://github.com/내Username/저장소이름.git`

메모장에 잠깐 붙여 넣어 “복사됐는지”만 확인해도 됩니다.

비밀번호·토큰을 책에 적지 마세요. **저장소 URL만**이면 됩니다.

[화면: Code 드롭다운 — HTTPS 선택 + URL + Copy 아이콘 강조]

여기서 막히면

“SSH / HTTPS 중 뭘 고르나요?” → **HTTPS**.

이 책은 Desktop 로그인(또는 HTTPS)으로 인증합니다.

SSH는 “키 파일을 따로 만드는” 다른 길이라 **이 장에서 다루지 않습니다**.

“Code 버튼이 안 보여요” → Code **탭**(파일 목록 화면)인지 확인하세요.

Settings·Actions 등 다른 탭에 있으면 Code로 돌아갑니다.

5. (기본 경로) GitHub Desktop으로 클론하기

초보에게 가장 덜 막히는 경로입니다.

설치 → 로그인 → 클론 → 폴더 위치 확인 순서만 따라가면 됩니다.

5-1. Desktop 설치 (아직 없다면)

1. 브라우저에서 GitHub Desktop 공식 안내로 이동합니다.
보통 github.com 도움말·다운로드 페이지에서 **Desktop** 을 찾을 수 있습니다.
(검색어 예: GitHub Desktop — 공식 다운로드인지 확인하세요.)
2. 본인 OS(Windows / macOS 등)용 설치 파일을 받습니다.
3. 설치 안내에 따라 설치합니다.
4. 앱을 실행합니다.

이미 설치되어 있으면 **5-2**로 건너뛰세요.

[화면: GitHub Desktop 첫 실행 — Welcome / Sign in 비슷한 화면]

여기서 막히면

“설치가 막혀요 / 관리자 비밀번호를 묻는다” →

본인 PC가 아니면 설치 권한이 없을 수 있습니다.

권한이 있는 컴퓨터에서 하거나, **7절**의 터미널 안내를 **참고만** 하세요.

이 책의 화면 캡처·단계는 **Desktop** 기준입니다.

5-2. GitHub 계정으로 Sign in

1. Desktop에서 **Sign in to GitHub.com**(또는 비슷한 문구)을 누릅니다.
2. 브라우저가 열리면 **2장에서 만든 계정**으로 로그인합니다.
3. “Desktop을 허용할까요?” 비슷한 권한이 나오면 **Authorize / Continue** 쪽으로 진행합니다.
4. Desktop으로 돌아오면 **로그인된 상태**가 됩니다. (아바타·Username이 보이면 OK)

[화면: Desktop — Sign in 성공 후 계정 표시(실명·얼굴 없이)]

여기서 막히면

“브라우저만 열고 Desktop이 안 바뀐다” →

브라우저 탭의 허용을 끝까지 눌렀는지 확인하고, Desktop을 한 번 앞으로 가져옵니다.

그래도 안 되면 Desktop을 종료 후 다시 실행 → Sign in을 한 번 더 시도하세요.

5-3. 저장소 클론 — 방법 두 가지 중 하나

아래 **A** 또는 **B** 중 **하나만** 하면 됩니다.

방법 A — Desktop 메뉴에서 클론

1. Desktop 상단 메뉴 **File** → **Clone repository...**(저장소 클론) 비슷한 항목을 클릭합니다.
2. **GitHub.com** 탭에서 본인 저장소 목록이 보이면, 3장에서 만든 저장소를 고릅니다.
3. **Local path**(저장 위치)를 확인합니다.
 - 기본값이 Documents/GitHub 비슷한 폴더인 경우가 많습니다.

- 어디에 생기는지 기억해 두세요. (나중에 파일 탐색기로 엽니다.)
4. **Clone** 버튼을 누릅니다.

방법 B — 브라우저 Code → Open with GitHub Desktop

1. github.com 저장소에서 **Code** 를 엽니다.
2. **Open with GitHub Desktop**(Desktop으로 열기) 비슷한 버튼을 누릅니다.
3. Desktop이 열리며 클론 창이 나오면, **Local path** 를 확인하고 **Clone** 을 누릅니다.

[화면: Desktop Clone repository 창 — 저장소 선택 + Local path + Clone 버튼]

클론이 끝나면 Desktop 가운데에 파일 목록(또는 “아직 변경 없음” 비슷한 상태)이 보입니다.
에러 없이 열리면 **1차 성공**입니다.

여기서 막히면

“목록에 내 저장소가 없다” →

1. Desktop에 **같은 계정**으로 로그인했는지
2. github.com에서는 저장소가 보이는지
3. **URL** 탭에 4절에서 복사한 `https://...\.git` 주소를 붙여 넣는 방법

“이미 있다 / 폴더가 비어 있지 않다” →

Local path 안의 **폴더 이름**이 이미 있으면 충돌할 수 있습니다.

경로를 다른 빈 위치로 바꾸거나, 예전 연습 폴더 이름을 정리한 뒤 다시 시도하세요.

(중요한 파일을 지우지 마세요.)

6. 클론이 끝난 뒤 — 폴더가 어디 있는지 확인

“복사가 됐다”고 느껴려면, **파일 탐색기**(또는 **Finder**) 에서 폴더를 한 번 열어 보는 것이 좋습니다.

단계

1. Desktop에서 **Repository** → **Show in Explorer**(Windows) / **Show in Finder**(macOS)
비슷한 메뉴를 누릅니다.
(문구가 Open in ... 쪽으로 보일 수도 있습니다.)
2. 열린 폴더 이름이 **저장소 이름**과 같은지 확인합니다.
3. 그 안에 **README.md** 파일이 있는지 확인합니다.
4. (선택) README.md를 메모장·기본 편집기로 열어, github.com에서 본 내용과 비슷한지 봅니다.
지금은 수정하지 않아도 됩니다. 수정·커밋은 **5장**입니다.

경로 예(실제 경로는 본인 Local path 기준):

- Windows: C:\Users\내이름\Documents\GitHub\hello-github
- macOS: /Users/내이름/Documents/GitHub/hello-github

예시는 가짜입니다. Desktop에 표시된 **Local path** 가 정답입니다.

[화면: 파일 탐색기 — 클론된 폴더 안 README.md가 보임(경로만, 실명 가림)]

확인 체크 (이 장 성공 기준)

- ☐ Desktop(또는 클론에 쓴 도구)에 **해당 저장소**가 열려 있다
- ☐ 로컬 폴더 경로를 **말할 수 있다** (또는 메모해 두었다)
- ☐ 폴더 안에 **README.md** 가 있다

세 가지면 이 장 실습은 충분합니다.

여기서 막히면

“README가 없어요” →

- github.com 저장소에 README가 있는지(3장)
- 클론한 저장소 이름이 맞는지(다른 빈 저장소를 연 것은 아닌지)
- 숨김 파일만 보고 있는 것은 아닌지

“폴더를 지워도 되나요?” →

연습 중이라도, **지금 단계에서는 남겨 두세요.**

5장에서 이 폴더의 파일을 고칩니다.

지웠다면 Desktop에서 **다시 클론**하면 됩니다. (github.com 원본은 그대로인 경우가 많습니다.)

7. (참고) 터미널 `git clone` — 이 책은 Desktop 중심

터미널에 익숙한 분은 아래처럼 클론할 수도 있습니다.

필수가 아닙니다. 이 책의 화면 따라하기는 **GitHub Desktop** 을 기준으로 합니다.

```
git clone https://github.com/내Username/저장소이름.git
```

- 위 명령은 **Git이 이미 설치된** 환경에서 동작합니다.
- 실행한 위치(현재 폴더) 아래에 저장소 폴더가 생깁니다.
- 인증 창·토큰을 물을 수 있습니다. 비밀번호를 책에 적지 마세요.

설치·PATH·인증 토큰 심화는 **이 책 범위 밖**입니다.

막히면 Desktop 경로(5~6절)로 돌아오세요.

8. 이 장에서 하지 않는 것

안 함	언제 / 왜
커밋(Commit)	5장
푸시(Push)	6장
PR	7장

안 함	언제 / 왜
SSH 키 만들기	나중에·선택. 이 장은 HTTPS + Desktop 로그인
Download ZIP만으로 연습 이어가기	커밋·푸시 연결이 약함 → 클론 권장
Actions	이 책 범위 밖
VS Code 심화 설정	필수 아님. 폴더만 확인하면 됨

기본 경로: **github.com Code(HTTPS) + GitHub Desktop 클론 + 로컬 폴더 확인**

9. 4장 체크리스트

- ☐ 클론을 “원격 저장소를 내 컴퓨터로 통째로 복사” 라고 말할 수 있다
- ☐ ZIP 받기와 클론의 차이를 한 줄로 안다
- ☐ **Code** → **HTTPS** → URL 복사를 해 봤다
- ☐ **GitHub Desktop**에 로그인했다 (또는 클론에 쓴 도구에 로그인했다)
- ☐ 저장소를 **Clone** 했다
- ☐ 로컬 폴더 위치를 확인했고, 그 안에 **README.md** 가 있다
- ☐ README를 고치거나 커밋하지 **않고** 여기까지 멈췄다 (다음 장용)

모두 체크되면 5장으로 갑니다.

여기서 막히면 (4장 전용 Q&A)

Q. Private 저장소도 클론되나요?

A. 본인 계정으로 로그인한 Desktop(또는 권한이 있는 방식)이면 되는 경우가 많습니다.
 목록에 안 보이면 URL을 직접 붙여 넣고, github.com에서 같은 계정으로 보이는지 확인하세요.

Q. 클론을 두 번 하면 어떻게 되나요?

A. 같은 경로에 다시 만들려 하면 “이미 있다”고 막히는 경우가 많습니다.
 이미 폴더가 있으면 그 폴더를 열어 쓰면 됩니다. 굳이 두 번 받을 필요 없습니다.

Q. Desktop 없이 브라우저만으로 끝나나요?

A. 브라우저만으로는 “내 PC 폴더에 Git 연결까지”가 이 책 범위에서 어렵습니다.
Desktop(권장) 또는 터미널 `git clone`(참고)이 필요합니다.

Q. Open with GitHub Desktop을 눌렀더니 앱 설치 안내만 나와요.

A. Desktop이 아직 없는 상태입니다. 5-1 설치 후 다시 시도하세요.

Q. 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다. **Code** → **HTTPS** → **Clone / Desktop**으로 열기 → **폴더 확인** 흐름만 같으면 같은 단계입니다. 비슷한 문구의 버튼을 누르세요.

Q. 이제 파일을 고쳐도 되나요?

A. 열어 보는 것은 OK입니다. **저장·커밋**은 5장에서 같이 합니다.

지금 고치면 “커밋을 안 한 변경”이 생겨 Desktop에 Changes가 보일 수 있습니다. 겁먹을 필요는 없고, 5장에서 이어서 하면 됩니다.

4장을 마치며

1. 클론 = **GitHub 저장소를 내 컴퓨터로 통째로 복사**
2. **ZIP ≠ 클론** — 한 바퀴 연습에는 클론
3. **Code → HTTPS → 복사** 위치를 안다
4. **GitHub Desktop** 로그인 후 **Clone** (또는 Code → Open with Desktop)
5. **Local path** 와 폴더 안 **README.md** 확인
6. **커밋·푸시**는 다음 장 — 지금은 “내 PC에 사본이 생겼다”까지

다음 5장에서는 이 로컬 폴더의 **README**를 한 줄 고친 뒤,

커밋(Commit) 으로 “이 시점의 변경을 기록”합니다.

아직 github.com에 자동으로 올라가지 않는다는 점도 중요합니다. (올리는 것은 6장 푸시)

[화면: 5장 예고 — Desktop Changes / Commit message 입력란(아직 누르지 말 것)] ## 따라 하기 체크 (4장)

- ☐ 영상에서 본 Code → HTTPS를 내 저장소에서 복사했다
 - ☐ Desktop(또는 안내 방법)으로 클론 완료했다
 - ☐ Local path 폴더에 **README.md**가 보인다
 - ☐ 다음: 그 폴더에서 **커밋**(5장) — 아직 푸시 X
-

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

5장. 커밋(Commit) — 변경을 기록하기

이 장에서는 4장에서 클론해 둔 로컬 폴더의 README.md 를
한두 줄만 고친 뒤, GitHub Desktop에서 커밋(Commit) 까지 합니다.

나두 영상 보고 오셨다면

“고친 뒤 → Changes에서 체크 → 메시지 쓰고 Commit” 흐름만 다시 손으로 따라가면 됩니다.

푸시(Push)는 아직 하지 않습니다.

커밋만 하면 내 컴퓨터 기록에만 남고, github.com에는 아직 안 올라간 상태입니다.
올리는 것은 6장입니다.

화면 문구는 Desktop·편집기가 가끔 바꿉니다.
버튼 글자가 조금 달라도 비슷한 문구면 그 버튼을 누르면 됩니다.

학습 목표

- 커밋 = “이 시점의 변경을 한 묶음으로 기록하기” 임을 이해한다
- 파일을 조금 고친 뒤, 스테이징(선택·체크) 하고 커밋할 수 있다
- 짧은 커밋 메시지를 쓰는 감을 익힌다
- “커밋만 함 ≠ github.com에 올라감” 을 구분한다 (푸시는 6장)

1. 커밋이란? (한 줄 + 비유)

커밋은 영어로는 **Commit** 입니다.

한 줄로 기억하면 됩니다.

커밋 = 이 시점의 변경을 한 묶음으로 기록하기

사진 앨범에 “오늘 찍은 사진 묶음”에 날짜·한 줄 메모를 붙이는 느낌에 가깝습니다.
파일을 고쳤다고 자동으로 기록이 생기지는 않습니다.
“이 변경을 기록할게” 하고 커밋을 눌러야 이력이 생깁니다.

단계	어디에 남나	이 장에서?
파일 수정·저장	내 PC 폴더	✓
커밋	내 PC의 Git 기록	✓
푸시	github.com 원격 저장소	✗ → 6장

1장에서 말한 한 바퀴 중, 지금은 「커밋」 단계입니다.

계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

[화면: 커밋 비유 — PC 폴더 변경 → Commit 도장 → (아직) github.com은 그대로]

여기서 막히면

“3장에서 Initial commit 이 있던데요?” →

저장소를 만들 때 README를 켜 두면, GitHub가 첫 커밋을 대신 만들어 둔 경우가 많습니다.

이 장에서는 **여러분이 직접** 두 번째(또는 다음) 커밋을 만듭니다.

2. 스테이징이란? (초보용으로만)

Desktop 화면에는 **Changes**(변경) 목록과, 파일 옆 **체크박스**가 보이는 경우가 많습니다.

초보가 외울 말은 이것만이면 됩니다.

스테이징 = “이번 커밋에 넣을 파일을 고르기(체크하기)”

- 체크된 파일 → 이번 커밋에 **포함**
- 체크 안 된 파일 → 이번 커밋에 **안 넣음** (나중에 따로 커밋 가능)

연습에서는 보통 **고친 README 하나만** 보이므로,

전부 체크된 상태로 Commit 하면 됩니다.

여러 파일을 골라 나누는 심화는 **이 책에서 다루지 않습니다.**

[화면: Desktop Changes — README.md 옆 체크박스 ON]

여기서 막히면

“스테이징·stage·add 단어가 다 나와요” →

Desktop에서는 **체크박스**가 그 역할에 가깝습니다.

터미널 git add 명령은 **필수가 아닙니다.** 이 장은 Desktop 기준입니다.

3. 시작 전에 확인

다음이 준비되어 있으면 됩니다.

1. 4장에서 **클론한 저장소**가 Desktop에 열려 있다
2. 로컬 폴더 안에 **README.md** 가 있다
3. Desktop에 **같은 GitHub 계정**으로 로그인되어 있다
4. (권장) 파일 탐색기/Finder에서 그 폴더를 한 번 열어 본 적이 있다

저장소가 없으면 4장으로 돌아가 클론부터 하세요.

이 장은 **이미 클론된 폴더**가 있다고 가정합니다.

[화면: Desktop — 4장에서 클론한 저장소가 선택된 상태(변경 없음)]

여기서 막히면

“Desktop에 저장소가 안 보여요” →

상단 **Current repository** (또는 저장소 이름) 클릭 → 목록에서 4장 저장소를 고르세요.

목록에도 없으면 4장처럼 **다시 클론**하면 됩니다.

4. 실습 — README를 가볍게 고치기

커밋하려면 **변경이 하나** 있어야 합니다.

거창한 코드가 아니어도 됩니다. **한 줄만** 추가해도 충분합니다.

단계

1. Desktop에서 **Repository** → **Show in Explorer**(Windows) / **Show in Finder**(macOS)로 로컬 폴더를 엽니다.
2. 폴더 안 **README.md** 를 더블클릭해 엽니다.
 - 메모장, TextEdit, 기본 텍스트 앱이면 충분합니다.
 - VS Code가 있어도 **심화 설정은 하지 않아도** 됩니다.
3. 맨 아래(또는 빈 줄)에 **한 줄**을 추가합니다. 예:

연습: 5장에서 커밋해 보기

4. **저장**합니다. (Ctrl+S / Cmd+S 등)
5. 편집기를 닫아도 됩니다.

완벽한 문장일 필요 없습니다. **파일이 바뀌었다**는 사실만 있으면 됩니다.

[화면: 메모장(또는 기본 편집기) — README.md에 한 줄 추가·저장]

여기서 막히면

“README가 안 열려요 / 확장자 경고” →

.md 는 글자 파일입니다. “다른 프로그램으로 열기”에서 **메모장·텍스트 편집기**를 고르세요.

“고쳤는데 Desktop에 아무 변화가 없다” →

1. **저장**을 했는지
 2. Desktop이 **같은 폴더**의 저장소를 보고 있는지
 3. Desktop 창을 한 번 앞으로 가져왔는지
- 잠시 기다리면 Changes에 파일이 나타납니다.

5. Desktop에서 변경 확인하기

파일을 저장하면 Desktop 왼쪽에 **Changes**(변경) 가 생깁니다.

단계

1. Desktop을 앞으로 가져옵니다.
2. 왼쪽(또는 가운데) **Changes** 목록에 README.md 가 보이는지 확인합니다.
3. 파일을 클릭하면 오른쪽(또는 아래)에 **무엇이 바뀌었는지** 줄 단위로 보일 수 있습니다.
 - 초록/빨강 줄 = 추가·삭제된 부분(표시 방식은 버전에 따라 다름)
4. README.md 옆 **체크박스**가 켜져 있는지 확인합니다.
 - 꺼져 있으면 **켜 둡니다.** (이번 커밋에 넣기)

[화면: Desktop Changes — README.md 선택 + diff(추가한 한 줄) 하이라이트]

여기까지면 “고친 내용이 Desktop에 잡혔다” 단계입니다.

아직 **커밋 버튼을 누르지 않아도** 됩니다. 다음 절에서 메시지와 함께 합니다.

여기서 막히면

“Changes가 비어 있어요” →

4절에서 **저장**했는지, **클론한 그 폴더**의 README인지 다시 확인하세요.

다른 복사본·다운로드 ZIP 폴더를 고친 것은 Desktop에 안 잡힙니다.

“파일이 너무 많이 보여요” →

지금은 연습용이라 README.md 만 체크하고, 모르는 파일은 **체크를 끄세요.**

(실수로 연 임시 파일·개인 메모가 섞였을 수 있습니다.)

6. 커밋 메시지 쓰고 Commit 하기

커밋에는 **짧은 메모(커밋 메시지)** 가 붙습니다.

나중에 “그때 뭘 바꿨지?”를 읽는 제목입니다.

메시지 예시 (복붙해도 됨)

- README에 연습 한 줄 추가
- 5장 연습 커밋
- Update README

초보는 **한 줄, 지금 한 일만** 쓰면 됩니다.

긴 소셜·이모지 나열·비밀번호는 넣지 마세요.

단계

1. Desktop 아래(또는 왼쪽 하단) **Summary**(요약) / **Commit message** 입력란을 찾습니다.
2. 위 예시 중 하나(또는 비슷한 한 줄)를 입력합니다.
3. (선택) **Description** 칸이 있으면 비워 뒀도 됩니다.
4. **Commit to main**(또는 Commit to master / Commit to ... 브랜치 이름)
비슷한 **Commit** 버튼을 누릅니다.
5. Changes 목록이 **비거나**, “No local changes” 비슷한 상태가 되면 **성공**입니다.

브랜치 이름이 main 이든 master 이든, 이 장에서는 **그 버튼을 누르면 됩니다.**
브랜치를 새로 만들거나 이름을 바꾸는 것은 **7장·이후**입니다.

[화면: Desktop — Summary에 “README에 연습 한 줄 추가” + Commit to main 버튼]

여기서 막히면

“Commit 버튼이 회색(비활성)이에요” →

1. Changes에 **체크된 파일**이 있는지
 2. Summary에 **글자가 하나라도** 있는지
- 둘 다 채우면 활성화되는 경우가 많습니다.

“이름을 설정하라 / Configure Git” 창이 뜬다 →

Desktop이 **커밋 작성자 이름·이메일**을 물을 수 있습니다.

github.com에 쓰는 **Username·가입 이메일**을 넣으면 무난합니다.

실명·얼굴 사진은 필수가 아닙니다. (2장에서 익명 OK였던 것과 같음)

“실수로 메시지를 이상하게 썼어요” →

연습 단계에서는 **한 번 더 README를 고치고 새 커밋**을 해도 됩니다.

메시지 고치기(amend)·되돌리기 심화는 **이 책 범위 밖**입니다.

7. 커밋만 된 상태 — github.com에는 아직 없음

여기가 이 장에서 **가장 중요한 구분**입니다.

	커밋 직후 (지금)	푸시 후 (6장)
내 PC Desktop / 로컬 기록	✅ 커밋이 보임	✅
github.com 저장소 페이지	❌ 아직 예전 내용일 수 있음	✅ 새 내용·커밋이 보임
인터넷의 다른 사람	보통 안 보임	Public이면 보일 수 있음

Desktop 상단에 **Push origin**(또는 Push) 버튼·배지가 보여도

이 장에서는 누르지 마세요.

눌러도 “망가지는” 일은 드물지만, 이 책은 **커밋과 푸시**를 장으로 나눕니다.

확인만 하고 싶다면:

1. 브라우저에서 본인 저장소(<https://github.com/내Username/저장소이름>)를 엽니다.
2. README를 봅니다.
3. **아직 연습 한 줄이 없으면 정상**입니다. (푸시 전)

4. Desktop **History**(기록) 탭에는 **방금 만든 커밋**이 보일 수 있습니다.
— 로컬에만 있는 기록입니다.

[화면: 왼쪽 Desktop History에 새 커밋 / 오른쪽 github.com README는 아직 예전 — 대비]

여기서 막히면

“github.com에도 바로 반영됐는데요?” →

드물게 다른 경로로 올렸거나, 예전에 같은 내용을 이미 푸시한 경우일 수 있습니다.

이 장 목표는 **커밋을 스스로 한 번** 하는 것입니다.

다음은 6장에서 **의도적으로 푸시**하고 웹에서 확인합니다.

“Push를 이미 눌러 버렸어요” →

괜찮습니다. 6장에서 “웹에서 확인하기”만 이어서 하면 됩니다.

겁먹고 되돌리려 하지 마세요. (되돌리기 심화는 범위 밖)

8. 연습 미션 (이 장 핵심)

아래를 **한 번에** 해 보세요. 체크하면 이 장 실습 완료입니다.

미션: 로컬에서만 커밋 한 번

- ☐ Desktop에서 4장 클론 저장소를 연다
- ☐ 로컬 README.md 에 **한 줄**을 추가하고 **저장**한다
- ☐ Desktop **Changes**에 README.md 가 보인다
- ☐ 파일 **체크(스테이징)** 가 켜져 있다
- ☐ Summary에 **짧은 메시지**를 쓴다
- ☐ **Commit** 버튼을 누른다
- ☐ Changes가 비었거나 “변경 없음”이 된다
- ☐ **Push**는 **누르지 않는다** (또는 이미 눌렀다면 6장으로 넘긴다)

두 번째 연습이 궁금하면, README에 다른 한 줄을 더 넣고 **커밋을 한 번 더** 해도 됩니다.
푸시는 여전히 **6장**까지 미뤄 두세요.

[화면: 미션 완료 — Desktop Changes 비어 있음 + History에 커밋 1개 이상]

9. 이 장에서 하지 않는 것

안 함	언제 / 왜
푸시(Push)	6장
PR	7장
브랜치 새로 만들기	7장에서 최소만

안 함	언제 / 왜
git add / git commit 터미널 필수화	Desktop으로 충분
커밋 수정(amend)·되돌리기·리셋	Git 심화 → 범위 밖
Actions	이 책 범위 밖
VS Code 심화 설정	텍스트 저장만 되면 됨

기본 경로: 로컬 README 수정 → Desktop Changes → 메시지 → Commit (푸시 없음)

10. 5장 체크리스트

- ☐ 커밋을 “이 시점의 변경을 한 묶음으로 기록” 이라고 말할 수 있다
- ☐ 스테이징을 “이번 커밋에 넣을 파일 체크” 정도로 이해한다
- ☐ 로컬 README.md 를 고치고 저장했다
- ☐ Desktop **Changes**에서 변경을 확인했다
- ☐ 짧은 커밋 메시지를 쓰고 **Commit** 했다
- ☐ 커밋 ≠ 푸시 를 구분한다 (웹에 안 올라도 정상일 수 있음)
- ☐ **Push origin** 은 6장으로 미뤘다 (또는 실수로 눌렀다면 6장에서 확인만 한다)

모두 체크되면 6장으로 갑니다.

나두 영상 보고 오셨다면 — 끝 점검

Changes → 체크 → 메시지 → Commit → **Push는 다음에**. 여덟 칸 미션만 채워도 충분합니다.

여기서 막히면 (5장 전용 Q&A)

Q. 커밋 메시지에 한글이 되나요?

A. 됩니다. 연습은 한글이 읽기 쉽습니다. 팀마다 영어만 쓰는 규칙이 있을 수 있지만, 이 책 연습은 한글 OK 입니다.

Q. main이 아니라 master로 나와요.

A. 저장소마다 기본 브랜치 이름이 다를 수 있습니다.

Commit to ... 에 적힌 이름으로 커밋하면 됩니다. 이름을 지금 바꾸지 마세요.

Q. “커밋할 게 없다”고만 나와요.

A. 파일이 저장되지 않았거나, 이미 커밋까지 끝난 상태입니다.

README를 한 글자라도 더 고치고 저장한 뒤 Changes를 다시 보세요.

Q. github.com의 README를 웹에서 직접 고쳐도 되나요?

A. 가능하지만, 이 책 경로는 로컬에서 고치고 커밋 → (다음) 푸시 입니다.

웹에서만 고치면 Desktop과 내용이 어긋날 수 있어, 지금은 로컬 연습을 권합니다.

Q. 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다. **고침 → Changes → 체크 → 메시지 → Commit** 흐름만 같으면 같은 단계입니다. **비슷한 문구**의 버튼을 누르세요.

Q. 이제 푸시해도 되나요?

A. **다음 장(6장)** 에서 같이 합니다.

버튼이 보여도 이 장은 “로컬 기록”까지가 목표입니다.

5장을 마치며

1. 커밋 = **이 시점의 변경을 한 묶음으로 기록**
2. 스테이징(초보) = **이번 커밋에 넣을 파일 체크**
3. README 한 줄 수정·저장 → Desktop **Changes** 확인
4. **짧은 메시지 + Commit**
5. **푸시는 아직** — github.com이 그대로여도 정상일 수 있음
6. History에 내 커밋이 보이면 이 장 완주

다음 **6장**에서는 Desktop의 **Push**(Push origin)로

이 커밋을 **github.com에 올리고**, 브라우저에서 README·커밋이 바뀌었는지 확인합니다.

[화면: 6장 예고 — Desktop Push origin 버튼(아직 누르지 말 것) + github.com 새로고침]

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

6장. 푸시(Push) — GitHub에 올리기

이 장에서는 5장에서 만든 로컬 커밋을
GitHub Desktop의 **Push**(Push origin)로 **github.com**에 올립니다.

올린 뒤 브라우저에서 **README·커밋이 바뀌었는지** 확인합니다.

PR(Pull Request)은 하지 않습니다. (7장)

나두 영상 보고 오셨다면

“Commit 다음 → Push → 웹에서 새로고침”만 손으로 한 번 따라가면 됩니다.

화면 문구는 Desktop·github.com이 가끔 바뀝니다.

버튼 글자가 조금 달라도 **비슷한 문구면** 그 버튼을 누르면 됩니다.

학습 목표

- 푸시 = “내 컴퓨터의 커밋을 GitHub 원격 저장소에 보내기” 임을 이해한다
- 푸시 후 **github.com**에서 파일이 바뀐 것을 확인할 수 있다
- 푸시가 실패했을 때 **먼저 볼 곳**(로그인·권한·인터넷)을 안다
- **클론 → 커밋 → 푸시** 한 바퀴를 말로 설명할 수 있다

1. 푸시란? (한 줄 + 표)

푸시는 영어로는 **Push** 입니다.

한 줄로 기억하면 됩니다.

푸시 = 내 컴퓨터에 있는 커밋을, GitHub(원격)로 보내기

단계	어디에 남나	이 장에서?
클론	내 PC 폴더	(4장에서 함)
커밋	내 PC Git 기록	(5장에서 함)
푸시	github.com	✓

5장 직후라면 Desktop에 **Push origin**(또는 Push) 배지·버튼이 보일 수 있습니다.
그게 “올릴 커밋이 로컬에 남아 있다”는 신호에 가깝습니다.

[화면: 푸시 비유 — PC History의 커밋 → 화살표 → github.com 저장소]

여기서 막히면

“이미 github.com에 반영돼 있어요” →

5장에서 Push를 실수로 눌렀거나, 예전에 같은 내용을 올린 경우일 수 있습니다.
이 장은 **웹에서 한 번 확인**하는 것까지가 목표입니다. 2~3절로 건너뛰어도 됩니다.

2. 시작 전에 확인

다음이 준비되어 있으면 됩니다.

1. 5장에서 **Commit** 까지 끝냈다 (Changes가 비어 있거나 “변경 없음”)
2. Desktop에 **같은 GitHub 계정**으로 로그인되어 있다
3. 인터넷이 된다
4. 브라우저에서 본인 저장소 주소를 열 수 있다
(<https://github.com/내Username/저장소이름>)

커밋이 아직 없다면 5장으로 돌아가 **한 줄 수정 → Commit** 부터 하세요.
이 장은 **올릴 로컬 커밋이 있다**고 가정합니다.

[화면: Desktop — History에 5장 커밋 + 상단 Push origin 배지]

여기서 막히면

“Push 버튼이 안 보여요 / 0 commits to push” →
올릴 커밋이 없거나, 이미 푸시가 끝난 상태입니다.
README에 **한 글자** 더 넣고 5장처럼 **다시 Commit** 한 뒤 돌아오세요.

3. Desktop에서 Push origin 하기

기본 경로는 **GitHub Desktop** 입니다.

단계

1. Desktop을 열고, 4~5장에서 쓰던 **저장소**가 선택돼 있는지 확인합니다.
2. 상단(또는 눈에 띄는 위치)의 **Push origin**
(또는 **Push** / Push 1 commit to origin 비슷한 문구)를 찾습니다.
3. 그 버튼을 **한 번** 누릅니다.
4. 잠시 기다립니다.
 - 진행 표시·스피너가 잠깐 보일 수 있습니다.
5. 배지 숫자(예: 1)가 사라지거나,
“Already up to date” / 푸시할 것 없음에 가까우면 **성공**에 가깝습니다.

브랜치 이름이 main 이든 master 이든,
지금은 **Push** 비슷한 버튼을 누르면 됩니다.
새 브랜치를 만들거나 PR을 여는 것은 **7장**입니다.

[화면: Desktop — Push origin 버튼(또는 Push 1 commit) 강조]

여기서 막히면

“Authentication failed / Sign in 창” →

Desktop에서 **GitHub 계정에 다시 로그인**하세요. (Preferences / Options → Accounts 비슷한 곳)

브라우저에만 로그인돼 있고 Desktop은 로그아웃된 경우가 많습니다.

“Permission denied / 권한이 없다” →

본인이 만든 저장소인지, 같은 Username으로 Desktop에 로그인했는지 확인하세요.

다른 사람 저장소에는 보통 푸시할 수 없습니다.

“네트워크 / Could not connect” →

Wi-Fi·유선·회사망 제한을 먼저 보세요. 잠시 후 **다시 Push** 하면 되는 경우도 많습니다.

“Fetch / Pull 을 먼저 하라”는 메시지 →

웹에서 README를 직접 고친 뒤일 수 있습니다.

초보 연습에서는 **웹에서 같은 파일을 고치지 않은** 상태로 다시 Push를 시도하세요.

Pull·병합 심화는 **이 책 범위 밖**(8장 Q&A에서 방향만)입니다.

4. github.com에서 확인하기

푸시는 “보냈다”에서 끝나지 않습니다.

웹에서 보이는지까지 확인하면 한 바퀴가 닫힙니다.

단계

1. 브라우저에서 본인 저장소를 엽니다.
`https://github.com/내Username/저장소이름`
2. **새로고침**(F5 / Cmd+R) 합니다.
3. **README** 미리보기에 5장에서 넣은 **연습 한 줄**이 보이는지 확인합니다.
4. (선택) 커밋 목록·히스토리 쪽으로 들어가
5장에서 쓴 **커밋 메시지**가 보이는지 봅니다.
 - 저장소 홈의 최근 커밋 한 줄, 또는
 - 시계/History 비슷한 아이콘·**Commits** 목록
5. Desktop **History**의 최신 커밋과 **제목이 같으면** 성공입니다.

확인	기대
README 본문	로컬에서 넣은 한 줄이 웹에도 있음
최근 커밋 메시지	5장 Summary와 같거나 비슷
Desktop Push 배지	없거나 올릴 것 없음

[화면: github.com README에 연습 한 줄 + 최근 커밋 메시지 하이라이트]

여기서 막히면

“푸시는 됐는데 README가 예전이에요” →

1. 다른 저장소 페이지를 연 건 아닌지
2. Username / 저장소 이름이 Desktop과 같은지
3. 브라우저 강력 새로고침 또는 시크릿 창을 순서대로 보세요.

“커밋은 보이는데 파일 내용이 없다” →

커밋에 README가 포함됐는지(5장에서 체크·Commit)부터 되짚으세요.

필요하면 README를 한 글자 더 고치고 **Commit** → **Push**를 한 번 더 하면 됩니다.

5. 실패하면 먼저 볼 곳 (3가지)

에러 문장이 길어도, 초보는 아래 세 줄만 먼저 점검하세요.

순서	점검	할 일
1	로그인	Desktop이 github.com 같은 계정 인지. 아니면 Sign in 다시
2	권한	내가 만든(또는 쓰기 권한 있는) 저장소인지
3	인터넷	연결·방화벽. 잠시 후 Push 재시도

그래도 안 되면 8장 Q&A·공식 도움말을 보면 됩니다.

지금은 “**겁먹고 설정 전체를 건드리지 않기**”가 더 중요합니다.

[화면: 점검 3가지 아이콘 — 로그인 / 권한 / 인터넷]

6. 말로 한 바퀴 — 클론 → 커밋 → 푸시

다음은 소리 내어(또는 메모에) 한 번 말해 보세요.

1. 클론 — GitHub 저장소를 내 컴퓨터 폴더로 복사해 온다.
2. 커밋 — 파일을 고친 뒤, 그 변경을 내 컴퓨터 기록으로 남긴다.
3. 푸시 — 그 기록을 GitHub에 보내, 웹에서도 같게 만든다.

한 줄 요약:

가져와(클론) → 기록하고(커밋) → 올린다(푸시).

1장에서 말한 한 바퀴 중, 지금은 「푸시」까지입니다.

계정 → 저장소 → 클론 → 커밋 → **푸시** → (다음) PR

[화면: 한 바퀴 다이어그램 — 클론 → 커밋 → 푸시 강조, PR은 흐리게]

7. 연습 미션 (이 장 핵심)

아래를 **한 번에** 해 보세요. 체크하면 이 장 실습 완료입니다.

미션: 로컬 커밋을 GitHub에 올리고 웹에서 확인

- ☐ Desktop에서 5장 커밋이 있는 저장소를 연다
- ☐ **Push origin**(또는 Push 비슷한 버튼)을 누른다
- ☐ 오류 없이 끝나거나, 올릴 커밋이 없어진다
- ☐ 브라우저에서 본인 저장소를 **새로고침**한다
- ☐ README에 **연습 한 줄**이 보인다
- ☐ (선택) 최근 **커밋 메시지**가 5장과 같다
- ☐ **PR은 만들지 않는다** (7장)

두 번째 연습이 궁금하면, README에 다른 한 줄을 넣고
Commit → Push → 웹 새로고침을 한 바퀴 더 돌려도 됩니다.

[화면: 미션 완료 — Desktop Push 배지 없음 + github.com README 일치]

8. 이 장에서 하지 않는 것

안 함	언제 / 왜
PR(Pull Request)	7장
새 브랜치 만들기	7장에서 최소만
Pull / Fetch 심화·충돌 해결	범위 밖 (8장에서 방향만)
git push 터미널 필수화	Desktop으로 충분
Actions	이 책 범위 밖
force push·히스토리 다시 쓰기	위험·심화 → 안 함

기본 경로: **Desktop Push origin → github.com 새로고침 → README·커밋 확인**

9. 6장 체크리스트

- ☐ 푸시를 “내 컴퓨터 커밋을 GitHub로 보내기” 라고 말할 수 있다
- ☐ Desktop에서 **Push origin**(비슷한 버튼)을 눌렀다
- ☐ github.com에서 **README(또는 파일) 변경**을 확인했다

- ☐ 실패 시 **로그인·권한·인터넷**을 먼저 본다는 걸 안다
- ☐ **클론 → 커밋 → 푸시**를 한 문장으로 말할 수 있다
- ☐ **PR은 7장으로** 미뤘다

모두 체크되면 7장으로 갑니다.

나두 영상 보고 오셨다면 — 끝 점검

Push → 웹 새로고침 → README 같으면 성공. 일곱 칸 미션만 채워도 충분합니다.

여기서 막히면 (6장 전용 Q&A)

Q. Push와 Publish 뭐가 달라요?

A. 저장소를 처음 Desktop에만 두고 GitHub에 **새 원격**을 만들 때 Publish 비슷한 말이 나올 수 있습니다. 이 책 경로(4장에서 **이미 클론**)에서는 보통 **Push** 입니다. **비슷한 문구**로 “올리는” 버튼을 누르면 됩니다.

Q. origin이 뭔가요?

A. Desktop·Git이 부르는 “**연결된 GitHub 저장소 별명**”에 가깝습니다. 초보는 “**내 github.com 저장소 쪽**” 정도로만 이해하면 됩니다.

Q. Private 저장소도 푸시되나요?

A. **본인 계정·권한이 있으면** 됩니다.

웹에서 확인할 때도 **로그인된 브라우저**로 열어야 보일 수 있습니다.

Q. 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다. **Push → 웹 새로고침 → 내용 확인** 흐름만 같으면 같은 단계입니다.

Q. 이제 PR 해도 되나요?

A. **다음 장(7장)** 에서 같이 합니다.

Compare & pull request 배너가 보여도 이 장은 “**올리기·웹 확인**”까지가 목표입니다.

6장을 마치며

1. 푸시 = **로컬 커밋을 GitHub로 보내기**
2. Desktop **Push origin**(비슷한 버튼) 한 번
3. github.com **새로고침** → README·커밋 확인
4. 막히면 **로그인 → 권한 → 인터넷**
5. 말로: **클론 → 커밋 → 푸시**
6. PR은 **아직** — 7장

다음 7장에서는 간단한 **브랜치** 위에서 변경을 올린 뒤,

Pull Request(PR) 로 “**합쳐 주세요**”를 제안하는 화면까지 갑니다.

[화면: 7장 예고 — Compare & pull request 버튼(아직 누르지 말 것)]

7장. Pull Request(PR) — 변경을 제안하기

이 장에서는 **본인이 만든 저장소**에서
간단한 **브랜치**를 만들고, 그 위에서 **커밋·푸시**한 뒤
Pull Request(PR)를 열어 **Merge**까지 한 바퀴 돌립니다.

6장까지는 **main**(또는 **master**)에 바로 올리는 연습이었습니다.
이 장은 “**옆 줄에서 고쳐 보고 → 본줄기에 합쳐 주세요**”를 제안하는 화면입니다.

나두 영상 보고 오셨다면

“새 브랜치 → 고치고 Commit·Push → Compare & pull request → Merge”
만 손으로 한 번 따라가면 됩니다.

화면 문구는 Desktop·github.com이 가끔 바꿉니다.
버튼 글자가 조금 달라도 **비슷한 문구**면 그 버튼을 누르면 됩니다.

다른 사람 저장소에 PR을 보내는 연습은 하지 않습니다.
초보가 안전하게 배우려면 **내 저장소에서 혼자 PR → Merge**가 가장 좋습니다.

학습 목표

- PR = “**변경을 본줄기(기본 브랜치)에 합쳐 달라는 제안**”임을 이해한다
- 간단한 **브랜치**를 만들고, 그 위에서 **커밋·푸시**한 뒤 PR을 연다
- PR **제목·설명**을 초보 수준으로 쓸 수 있다
- **Merge** = **제안이 받아들여져 합쳐진 것**임을 안다

1. PR이란? (한 줄 + 표)

PR은 영어로는 **Pull Request**입니다.
한국어로는 “**풀 리퀘스트**”, 줄여서 **PR**이라고 많이 부릅니다.
한 줄로 기억하면 됩니다.

PR = 내 변경을 본줄기(기본 브랜치)에 합쳐 달라고 제안하기

아직 “**합쳐진 상태**”가 아닙니다.
제안 글을 남기고, 나중에 **Merge** 하면 본줄기에 들어갑니다.

단계	한 줄	이 장에서?
브랜치	옆 줄(작업 줄)을 하나 만든다	✓
커밋·푸시	그 줄 위에서 고치고 올린다	✓

단계	한 줄	이 장에서?
PR	“본줄기에 합쳐 주세요” 제안	✓
Merge	제안이 받아들여져 합쳐짐	✓

1장에서 말한 한 바퀴의 **마지막**에 가깝습니다.

계정 → 저장소 → 클론 → 커밋 → 푸시 → **PR**

[화면: PR 비유 — 옆 줄(브랜치) 변경 → “합쳐 주세요” 카드 → 본줄기(main)]

여기서 막히면

“왜 main에 바로 안 올려요?” →

바로 올려도 되는 작은 연습은 6장까지로 충분합니다.

PR은 **협업·검토·실수 줄이기**를 위한 기본 습관입니다.

혼자 연습해도 **같은 버튼**을 익혀 두면 나중에 편합니다.

2. 브랜치란? (초보용으로만)

브랜치는 영어로는 **Branch** 입니다.

초보가 외울 말은 이것만이면 됩니다.

브랜치 = 본줄기와 나란히 둔 “작업용 줄”

- **본줄기**: 보통 이름이 main 또는 master (저장소의 기본 줄)
- **작업 줄**: 예) practice-pr — 여기서 파일을 고치고 커밋한다

이 책에서는 **이름 하나 만들고 그 위에서 작업**하는 것만 합니다.

브랜치 이름 규칙·삭제·리베이스·충돌 해결은 **다루지 않습니다**.

[화면: 브랜치 다이어그램 — main 직선 + practice-pr 옆 갈래]

여기서 막히면

“main이랑 master 중 뭐예요?” →

저장소마다 기본 이름이 다릅니다. Desktop·웹 상단에 보이는 **현재 브랜치 이름**을 따르면 됩니다.

PR을 열 때 **base(합쳐질 쪽)** 가 main/master 인지만 확인하면 됩니다.

3. 시작 전에 확인

다음이 준비되어 있으면 됩니다.

1. 6장까지 **클론 → 커밋 → 푸시**를 한 적이 있다
2. Desktop에 **본인 저장소**가 열려 있고, **같은 GitHub 계정**으로 로그인되어 있다

3. 인터넷이 된다
4. 브라우저에서 본인 저장소를 열 수 있다
(<https://github.com/내Username/저장소이름>)

푸시가 아직 안 되면 6장으로 돌아가 **Push** → **웹 확인**부터 하세요.
이 장은 **이미 푸시할 수 있다**고 가정합니다.

[화면: Desktop — 6장 저장소 선택, 현재 브랜치 main(또는 master)]

여기서 막히면

“다른 사람 저장소로 연습해도 되나요?” →

아니요. 이 장은 내가 만든(쓰기 권한 있는) 저장소만 씁니다.

남의 저장소에 PR을 보내는 흐름은 권한·Fork 등이 붙어 **초보 연습에 맞지 않습니다.**

4. Desktop에서 새 브랜치 만들기

기본 경로는 **GitHub Desktop** 입니다.

단계

1. Desktop을 열고, 연습용 **본인 저장소**를 선택합니다.
2. 현재 브랜치가 main(또는 master)인지 확인합니다.
 - 상단 **Current branch** (또는 브랜치 이름) 비슷한 곳을 봅니다.
3. **New branch**(또는 **Create branch** / 브랜치 이름 옆 · 메뉴)를 누릅니다.
4. 새 이름을 입력합니다. 예:

```
practice-pr
```

5. **Create branch**(또는 Create)를 누릅니다.
6. 상단 브랜치 이름이 **practice-pr** 로 바뀌었는지 확인합니다.

이제부터 고치는 내용은 **본줄기가 아니라 이 작업 줄**에 씁니다.

[화면: Desktop — New branch / Create branch 창에 practice-pr 입력]

여기서 막히면

“New branch가 안 보여요” →

상단 브랜치 이름(Current branch)을 **클릭**하면 목록·새 브랜치 버튼이 나오는 경우가 많습니다.

메뉴 **Branch** → **New Branch** 비슷한 경로도 있습니다.

“이름을 한글로 해도 되나요?” →

연습은 **영문·숫자·하이픈**(practice-pr)을 권장합니다.

공백·특수문자는 피하세요.

5. 브랜치 위에서 고치고 → 커밋 → 푸시

5~6장과 같은 버튼입니다.

다른 점은 **지금 브랜치가 practice-pr** 라는 것뿐입니다.

단계

1. Desktop에서 **Repository → Show in Explorer / Show in Finder** 로 폴더를 엽니다.
2. **README.md** 를 열어 **한 줄**을 추가합니다. 예:

연습: 7장 PR용 한 줄

3. 저장합니다.
4. Desktop **Changes**에서 파일이 보이는지 확인하고, 체크가 켜져 있으면 그대로 둡니다.
5. **Summary**(커밋 메시지)에 짧게 씁니다. 예:

Add practice line for PR

또는 한글:

7장 PR 연습 한 줄 추가

6. **Commit to practice-pr**(또는 Commit 비슷한 버튼)을 누릅니다.
7. 상단의 **Push origin**(또는 Publish branch / Push 1 commit 비슷한 문구)을 누릅니다.
 - 이 브랜치를 **처음** 올릴 때는 **Publish branch** 비슷한 말이 나올 수 있습니다.
 - “**원격에 이 줄을 올린다**” 뜻이면 같은 단계입니다.

[화면: Desktop — Commit to practice-pr 후 Push origin / Publish branch]

여기서 막히면

“Commit to main 이라고 나와요” →

아직 브랜치가 main입니다. **4절로 돌아가** practice-pr 를 만들고 다시 고치세요.

(이미 main에 커밋해 버렸다면, 웹에서 확인만 하고 **새 브랜치를 다시 만들어** 다른 한 줄로 연습해도 됩니다.)

“Publish branch 와 Push 중 뭐예요?” →

원격에 없는 **새 브랜치**를 처음 보낼 때 Publish 비슷한 말이 자주 나옵니다.

비슷한 문구로 “올리는” 버튼을 누르면 됩니다.

6. github.com에서 PR 열기

푸시(또는 Publish)가 끝나면, 브라우저에서 **제안을 글로** 남깁니다.

단계

1. 브라우저에서 본인 저장소를 엽니다.
`https://github.com/내Username/저장소이름`
2. 새로고침합니다.
3. 노란 배너나 안내로
Compare & pull request(또는 **Create pull request** 비슷한 문구)가 보이면 누릅니다.
4. 배너가 없으면:
 - 상단 **Pull requests** 탭 → **New pull request**
 - **base:** main(또는 master) ← 합쳐질 쪽
 - **compare:** practice-pr ← 내 작업 줄
 - **Create pull request**(또는 Compare) 비슷한 버튼을 누릅니다.
5. **제목(Title)** 과 **설명(Leave a comment)** 을 채웁니다. (아래 7절)
6. 초록(또는 강조) **Create pull request** 버튼을 누릅니다.

이제 PR **한 장**이 열린 상태입니다.

아직 main에 합쳐진 것은 **아닙니다**.

[화면: github.com — Compare & pull request 배너 / base main ← compare practice-pr]

여기서 막히면

“Compare & pull request 가 안 보여요” →

1. 브랜치를 푸시(Publish)했는지
2. 같은 저장소 페이지인지
3. **Pull requests** → **New pull request** 로 base/compare를 직접 고르기
순서로 보세요.

“Able to merge / Can't automatically merge” →

혼자 연습·한 줄 추가면 보통 **합칠 수 있음**에 가깝습니다.

“자동으로 합칠 수 없다”면 충돌 가능성이 있습니다.

초보 연습에서는 **새 브랜치 + README 다른 한 줄**로 다시 만드는 편이 쉽습니다.

충돌 해결은 **이 책 범위 밖**(8장에서 방향만)입니다.

7. PR 제목·설명 — 초보 수준이면 충분

거창한 템플릿은 필요 없습니다.

무엇을 바꿨는지만 짧게 쓰면 됩니다.

제목 예시

Add README practice line for chapter 7

또는:

설명 예시

연습용 PR입니다.
README에 한 줄을 추가했습니다.
본인 저장소에서 Merge까지 해 봅니다.

쓰면 좋은 것	안 해도 되는 것
무엇을 바꿨는지 한두 문장	긴 템플릿·체크리스트 전부
(선택) 왜 연습인지 한 줄	이슈 번호·리뷰어 멘션 필수화

[화면: Create pull request — Title·설명 입력란 + Create 버튼]

여기서 막히면

“영문으로 써야 하나요?” →

아니요. 초보 연습은 한글이어도 됩니다.

나중에 팀에 맞추면 됩니다.

“Draft / Ready for review 가 있어요” →

연습은 일반 **Create pull request** 로 열면 됩니다.

Draft(초안)는 “아직 검토 요청 전” 표시에 가깝습니다. 필수는 아닙니다.

8. Merge — 제안이 받아들여져 합쳐진 것

본인 저장소이므로, 스스로 Merge 해도 됩니다.

혼자 연습할 때 가장 안전한 방법입니다.

단계

1. 방금 만든 PR 페이지에 있습니다.
2. **Merge pull request**(또는 Confirm merge 비슷한 문구)를 찾습니다.
3. 버튼을 누른 뒤, 확인 단계가 있으면 **Confirm merge** 를 누릅니다.
4. **Pull request successfully merged**(또는 Merged 비슷한 표시)가 보이면 성공입니다.
5. (선택) **Delete branch** 안내가 나오면, 연습 브랜치를 지워도 됩니다.
 - 안 지워도 이 장 목표는 달성입니다.

한 줄로 기억하세요.

Merge = PR 제안이 받아들여져, 본줄기에 합쳐진 것

[화면: PR 페이지 — Merge pull request → Confirm merge → Merged]

여기서 막히면

“Merge 버튼이 비활성이에요” →

권한·보호된 브랜치·검사 대기일 수 있습니다.

본인이 만든 개인 연습 저장소인지 확인하세요.

회사·조직 저장소는 규칙이 다를 수 있어 **이 책 연습용으로 비추천**입니다.

“Merge 종류(Create a merge commit / Squash / Rebase)가 많아요” →

초보는 기본값 **Merge pull request**(merge commit) 그대로면 충분합니다.

Squash·Rebase 차이는 **심화**라 다루지 않습니다.

9. 합쳐진 뒤 확인하기

Merge 후에는 **웹에서 본줄기**를 한 번 보면 한 바퀴가 닫힙니다.

단계

1. 저장소 **Code** 탭(파일 목록)으로 이동합니다.
2. 브랜치가 main(또는 master)인지 확인합니다.
3. **README**에 연습: 7장 PR용 한 줄 이 보이는지 확인합니다.
4. (선택) Desktop에서 브랜치를 main으로 바꾼 뒤
Fetch origin / Pull origin 비슷한 버튼이 있으면 눌러 로컬도 맞출 수 있습니다.
 - Pull 심화·충돌은 이 장에서 **강제하지 않습니다**.
 - 웹 README만 같아도 이 장 실습 완료로 칩니다.

확인	기대
PR 상태	Merged
main의 README	연습 한 줄이 있음
(선택) Desktop	Fetch/Pull 후 로컬도 같음

[화면: github.com main README에 7장 연습 한 줄 + PR Merged 배지]

10. 말로 한 바퀴 — 브랜치 → PR → Merge

다음은 소리 내어(또는 메모에) 한 번 말해 보세요.

1. **브랜치** — 본줄기 옆에 작업 줄을 만든다.
2. **커밋·푸시** — 그 줄에서 고치고 GitHub에 올린다.
3. **PR** — “본줄기에 합쳐 주세요”라고 제안한다.
4. **Merge** — 제안이 받아들여져 본줄기에 합쳐진다.

한 줄 요약:

옆에서 고쳐 올리고(브랜치) → 합쳐 달라고 하고(PR) → 합친다(Merge).

[화면: 한 바퀴 다이어그램 — 브랜치 → 푸시 → PR → Merge 강조]

11. 연습 미션 (이 장 핵심)

아래를 한 번에 해 보세요. 체크하면 이 장 실습 완료입니다.

미션: 내 저장소에서 브랜치 → PR → Merge

- ☐ Desktop에서 본인 저장소를 연다
- ☐ 새 브랜치 practice-pr(또는 비슷한 이름)를 만든다
- ☐ README에 연습 한 줄을 넣고 **Commit** 한다
- ☐ **Push origin / Publish branch**(비슷한 버튼)로 올린다
- ☐ github.com에서 **Compare & pull request**(또는 New pull request)로 PR을 연다
- ☐ **제목·설명**을 짧게 쓰고 Create pull request 한다
- ☐ **Merge pull request** → Confirm 까지 한다
- ☐ main(또는 master) README에 한 줄이 **보이는지** 확인한다
- ☐ 다른 사람 저장소에는 PR을 보내지 않았다

두 번째 연습이 궁금하면 브랜치 이름만 practice-pr-2 로 바꿔 같은 미션을 한 바퀴 더 돌려도 됩니다.

[화면: 미션 완료 — PR Merged + main README 일치]

12. 이 장에서 하지 않는 것

안 함	언제 / 왜
다른 사람 저장소에 PR / Fork	권한·협업 심화 → 초보 연습에 비추천
브랜치 이름 규칙·삭제 전략	최소 생성만
충돌 해결·rebase·squash 필수화	범위 밖 (8장에서 방향만)
GitHub Actions / 자동 검사 설정	이 책 범위 밖
터미널 git checkout·git merge 필수화	Desktop + 웹으로 충분
force push·히스토리 다시 쓰기	위험·심화 → 안 함

기본 경로:

Desktop 새 브랜치 → Commit → Push/Publish → 웹에서 PR → Merge → README 확인

13. 7장 체크리스트

- ☐ PR을 “본줄기에 합쳐 달라는 제안” 이라고 말할 수 있다
- ☐ Desktop에서 새 브랜치를 만들었다
- ☐ 그 브랜치에서 커밋·푸시(또는 Publish) 했다
- ☐ github.com에서 PR을 열었다 (제목·설명 포함)
- ☐ Merge = 제안이 받아들여져 합쳐진 것임을 안다
- ☐ 본인 저장소에서 Merge까지 해 보았다
- ☐ Actions·충돌 심화는 미뤘다

모두 체크되면 8장으로 갑니다.

나두 영상 보고 오셨다면 — 끝 점검

브랜치 → Push → Compare & pull request → Merge → README 같으면 성공.
미션 아홉 칸만 채워도 충분합니다.

여기서 막히면 (7장 전용 Q&A)

Q. PR이랑 Merge가 같은 말인가요?

A. 아니요. PR은 제안, Merge는 그 제안이 합쳐진 결과에 가깝습니다.

Q. 꼭 브랜치가 있어야 PR을 열 수 있나요?

A. 보통 다른 줄(브랜치)의 변경을 본줄기에 제안하는 흐름입니다.
이 장 연습은 새 브랜치를 전제로 합니다.

Q. main에 바로 푸시한 다음에도 PR을 만들 수 있나요?

A. 같은 줄에 이미 들어간 변경은 “제안할 차이”가 없을 수 있습니다.
연습은 먼저 브랜치 → 그 위에서 커밋 순서를 지키세요.

Q. Close pull request 를 누르면요?

A. 제안을 합치지 않고 닫는 것에 가깝습니다.
연습 중 잘못 열었으면 Close 해도 됩니다. 다시 브랜치·PR을 만들면 됩니다.

Q. 화면이 책과 조금 달라요.

A. UI는 자주 바뀝니다.
브랜치 → 푸시 → PR 열기 → Merge → README 확인 흐름만 같으면 같은 단계입니다.

7장을 마치며

1. PR = 변경을 본줄기에 합쳐 달라는 제안
2. 브랜치에서 고치고 커밋·푸시
3. github.com에서 Compare & pull request(비슷한 버튼)
4. 제목·설명은 짧게

5. **Merge** = 제안이 받아들여져 합쳐진 것

6. **내 저장소**에서 혼자 연습하면 안전하다

다음 **8장**에서는 2~7장을 따라가다 **자주 막히는 장면을**

짧은 **Q&A**로 풀어 봅니다.

로그인·권한·“어디에 올렸지?”·영문 에러 문구를 덜 무섭게 만드는 장이 됩니다.

[화면: 8장 예고 — Q&A 목록 미리보기(로그인 / 권한 / 어디에 올렸는지)]

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

8장. 자주 막히는 장면 Q&A

2~7장을 따라가다 보면

로그인·권한·“어디에 올렸지?”·영문 문구에서 한 번씩 멈춥니다.

이 장은 새 기능을 배우지 않습니다.

막힌 자리에서 먼저 볼 곳만 짧게 짚습니다.

나두 영상 보고 오셨다면

영상과 버튼 글자가 달라도 괜찮습니다.

아래 Q에서 비슷한 문구를 고른 뒤, 먼저 볼 곳만 순서대로 확인하세요.

화면 문구는 Desktop.github.com이 가끔 바뀝니다.

글자가 조금 달라도 비슷한 의미면 같은 단계입니다.

불법 우회·타인 계정 침해·무단 접근 방법은 다루지 않습니다.

본인 계정·본인 저장소 기준입니다.

학습 목표

- 로그인·인증·권한 막힘을 스스로 점검할 수 있다
- “어디에 올렸는지 모르겠다”를 되짚는 순서를 안다
- 클론·푸시·PR에서 자주 보는 메시지를 덜 무섭게 읽는다
- 이 책 범위를 넘으면 다음에 무엇을 찾아볼지 방향을 안다

어떻게 쓰나 (30초)

1. 아래 그룹에서 지금 상황과 가까운 Q를 고른다
2. A의 “먼저 볼 곳”만 위에서부터 확인한다
3. 그래도 안 되면 이 장 끝 복구 체크리스트 → 범위 밖 표로 간다

[화면: 8장 지도 — 계정 / 어디에 올렸는지 / 클론 / 커밋·푸시 / PR / 범위 밖]

A. 계정 · 로그인 · 인증 · 권한

Q1. 로그인이 안 돼요. 비밀번호가 틀리다고 나와요.

A. 대부분 이메일(또는 Username) + 비밀번호 조합 문제입니다.

먼저 볼 곳

1. 주소가 <https://github.com> 인지 확인한다
2. **Sign in** 화면에서 Username / email 칸을 **가입할 때 쓴 것**과 맞춘다
3. 비밀번호를 **한 번 천천히** 다시 입력한다 (대소문자·한영)
4. 그래도 안 되면 공식 **Forgot password?**(비슷한 문구)로 **본인 이메일** 재설정만 한다
5. 2장으로 돌아가 **Sign out** → **Sign in** 한 번 더 연습한다

여기서 막히면

비밀번호·복구 메일·인증 코드는 **본인만** 다룹니다.
다른 사람에게 물어보거나 공유하지 마세요.

Q2. 이메일 인증(Verify)을 못 끝냈어요.

A. 계정이 “거의 만들어졌는데” 웹에서 기능이 막히는 흔한 경우입니다.

먼저 볼 곳

1. 가입에 쓴 **메일함**을 연다 (스팸·프로모션 함도)
2. GitHub 발신 메일에서 **Verify / Confirm**(비슷한 링크)를 누른다
3. 링크가 만료됐으면 github.com에 로그인한 뒤 **재발송** 안내를 따른다
4. 다른 브라우저·시크릿 창에서 같은 계정으로 다시 열어 본다

이메일·비밀번호를 바꾸는 “해킹” 류 조언은 **이 책 밖**입니다.
공식 화면의 재설정만 쓰세요.

Q3. Private 저장소가 안 보여요 / 푸시가 거절돼요.

A. 로그인·권한을 먼저 의심합니다. (인터넷은 그다음)

먼저 볼 곳

1. 브라우저가 **본인 계정**으로 로그인돼 있는지 (우측 상단 아바타)
2. 저장소 URL의 사용자명이 **내 Username**인지
3. Desktop도 **같은 계정**으로 로그인돼 있는지 (Preferences / Options → Accounts 비슷한 곳)
4. 저장소가 **Private**이면, 로그인 안 된 창·다른 계정 창에서는 안 보일 수 있다
5. 그래도 안 되면 6장 점검: **로그인** → **권한** → **인터넷**

타인 저장소에 몰래 올리거나 권한을 우회하는 방법은 **다루지 않습니다**.

Q4. Desktop과 브라우저 계정이 다른 것 같아요.

A. “웹에는 있는데 Desktop에 없다 / 푸시가 이상하다”의 단골 원인입니다.

먼저 볼 곳

1. 브라우저: github.com → 아바타 → 표시된 **Username**

2. Desktop: 계정(또는 Sign in to GitHub.com)에서 **같은 Username**인지
3. 다르면 Desktop에서 **Sign out** → **본인 계정으로 다시 Sign in**
4. 그다음 해당 저장소를 다시 연다 (필요하면 4장처럼 다시 클론)

[화면: 브라우저 Username vs Desktop Accounts — 같은 이름인지 비교(개인정보 가림)]

B. “어디에 올렸는지 모르겠다”

Q5. 커밋은 한 것 같은데, GitHub에 안 보여요.

A. 커밋 ≠ 푸시 입니다. (5장·6장)

먼저 볼 곳 (되짚는 순서)

1. **Desktop** Changes / History에서 최근 커밋이 **로컬에만** 있는지 본다
2. **Push origin**(비슷한 버튼)을 아직 안 눌렀으면 누른다
3. 브라우저에서 저장소를 **새로고침**한다
4. README(또는 고친 파일) 내용이 Desktop과 같은지 본다
5. (선택) 커밋 목록·최근 메시지에 5~6장 연습 문구가 있는지 본다

한 줄

로컬에만 있으면 → **푸시**. 푸시 후 웹이 안 바뀌면 → **새로고침·계정·저장소 URL**.

Q6. 저장소가 여러 개라 어떤 폴더인지 모르겠어요.

A. “올린 곳”과 “내 PC 폴더”를 **한 줄로 짝** 지으면 됩니다.

먼저 볼 곳

1. 브라우저 주소창: `https://github.com/내Username/저장소이름`
2. Desktop에서 그 저장소를 선택 → **Local path**(비슷한 항목) 경로를 적는다
3. 파일 탐색기(또는 Finder)로 그 경로를 연다
4. 연습용 README가 **그 폴더**에 있는지 확인한다
5. 엉뚱한 폴더를 고치고 있었다면, **맞는 저장소**를 Desktop에서 다시 연다

[화면: github.com URL + Desktop Local path를 나란히(경로만, 실명 가림)]

Q7. 파일을 고쳤는데 “변경 없음”처럼 보여요.

A. 다른 폴더를 열었거나, 저장(Save)을 안 했거나, 이미 커밋·푸시가 끝난 상태일 수 있습니다.

먼저 볼 곳

1. 에디터에서 **저장**했는지
2. Desktop이 **지금 연습 중인 저장소**를 보고 있는지

3. Changes에 파일이 안 뜨면 → 로컬 폴더의 파일이 정말 바뀌었는지
 4. 이미 Push까지 끝났다면 github.com과 내용이 **같아서** 변경이 없을 수 있다 — 정상
-

C. 클론에서 자주 막힘

Q8. Clone 버튼·URL을 어디에 두는지 모르겠어요.

A. 4장 흐름만 다시 탑니다.

먼저 볼 곳

1. 본인 저장소 페이지 → **Code**(비슷한 버튼)
2. **HTTPS** 탭인지 확인
3. URL 복사 또는 Open with GitHub Desktop(비슷한 문구)
4. Desktop이 뜨면 **Local path**를 확인한 뒤 Clone
5. 끝난 뒤 탐색기에서 폴더가 생겼는지 본다

SSH·터미널 git clone 심화는 **이 책에서 필수 아님** (참고만).

Q9. “이미 있다 / destination already exists” 비슷한 말이 나와요.

A. 같은 이름 폴더가 **Local path**에 이미 있을 때 자주 납니다.

먼저 볼 곳

1. Desktop Clone 창의 **Local path**를 확인한다
2. 그 경로에 **같은 이름 폴더**가 있으면 → 다른 상위 폴더를 고르거나, 연습용으로 **새 폴더 이름** 경로를 쓴다
3. 예전에 클론해 둔 것이면 → **새로 클론하지 말고** Desktop에서 그 저장소를 연다

기존 폴더를 함부로 삭제하는 안내는 하지 않습니다.

경로만 바꿔 클론하거나, 이미 있는 쪽을 여세요.

Q10. 클론은 됐는데 빈 것 같아요 / README가 없어요.

A. 3장에서 README를 안 켜 저장소이거나, 다른 저장소를 연 경우입니다.

먼저 볼 곳

1. 브라우저에서 같은 저장소에 **README**가 보이는지
 2. 없으면 웹에서 README를 추가하는 안내에 따라 **최소 파일**을 만든다 (3장 복습)
 3. Desktop에서 **Fetch / Pull**(비슷한 버튼)이 있으면 받아 본다 — 심화 충돌은 범위 밖
 4. Local path가 **방금 클론한 그 저장소**인지 다시 확인
-

D. 커밋 · 푸시에서 자주 막힘

Q11. Commit 버튼이 비활성(회색)이에요.

A. 보통 메시지 없음 또는 선택된 변경 없음입니다.

먼저 볼 곳

1. Changes에서 고친 파일 옆 **체크**가 켜져 있는지 (스테이징)
 2. Summary / Commit message 칸에 **짧은 한 줄**을 썼는지
 3. 이미 커밋이 끝났다면 버튼이 비는 것이 정상 → 다음은 **Push**
-

Q12. Push가 실패했어요. (인증·permission·network 비슷한 문구)

A. 6장과 같습니다. 로그인 → 권한 → 인터넷 순.

먼저 볼 곳

1. Desktop 계정이 **본인**인지 (Q4)
2. 그 저장소에 **푸시할 권한**이 있는 본인 repo인지
3. Wi-Fi·유선이 되는지, 잠깐 끊겼다가 다시 Push
4. 브라우저에서 같은 저장소가 **열리고 보이는지**
5. 문구에 *Authentication failed / Permission denied / Could not connect* 비슷한 말이 있으면 → 위 1~3을 반복

force push·히스토리 덮어쓰기·토큰을 남에게 묻는 방법은 다루지 않습니다.

Q13. “Push할 커밋이 없다” / 배지가 없어요.

A. 이미 다 올렸거나, 아직 커밋을 안 한 상태입니다.

먼저 볼 곳

1. Changes에 남은 수정이 있으면 → **Commit** 먼저 (5장)
 2. History에 최신 커밋이 있는데 Push 배지가 없으면 → 웹 **새로고침**으로 확인
 3. 웹과 로컬 README가 같으면 → **성공한 상태**일 수 있다
-

E. Pull Request에서 자주 막힘

Q14. Compare & pull request 버튼이 안 보여요.

A. 브랜치에 올린 변경이 없거나, 이미 PR을 열었거나, 본줄기와 차이가 없을 수 있습니다.

먼저 볼 곳

1. Desktop에서 **작업 브랜치**에 있는지 (예: practice-pr)
2. 그 브랜치에서 **커밋 → Push/Publish** 했는지 (7장)
3. github.com 저장소 → **Pull requests** 탭에 이미 열린 PR이 있는지
4. 없으면 브랜치 메뉴·배너에서 **Compare & pull request**(비슷한 문구)를 찾는다
5. 그래도 없으면: 브랜치에서 README를 **한 줄 더** 고치고 Commit → Push 후 새로고침

Q15. PR은 열렸는데 Merge가 안 돼요.

A. 이 책은 **본인 저장소**에서 혼자 Merge 연습이 목표입니다.

먼저 볼 곳

1. PR이 **Open** 상태인지 (Close 하지 않았는지)
2. **Merge pull request → Confirm merge**(비슷한 두 단계)인지
3. 빨간 충돌(conflict) 경고가 있으면 → **이 책 범위 밖** (아래 F)
4. Actions 체크가 돌아가는 화면이 보여도 → **심화·범위 밖** (무시하고 Merge 가능 여만 본다)
5. 본인 repo가 아니면 Merge 권한이 없을 수 있다 → **내 저장소**로 연습 (7장)

Q16. 잘못된 PR을 열었어요.

A. 합치지 않을 거면 **Close pull request**(비슷한 문구)로 닫을 수 있습니다.

먼저 볼 곳

1. 해당 PR 페이지에서 Close
2. 필요하면 새 브랜치 이름으로 7장 미션을 **다시** 한 바퀴
3. 이미 Merge까지 했다면 → 연습용 README 한 줄을 더 고치는 식으로 **다음 연습**으로 넘긴다 (되돌리기 심화는 범위 밖)

F. 기타 · 이 책 범위 밖이면

Q17. 화면에 영어가 많아서 무서워요.

A. 초보도 많이 그렇습니다. **단어 하나만** 고르세요.

비슷한 문구	초보용 뜻
Sign in / Sign out	로그인 / 로그아웃
Clone	복사해 오기
Commit	기록하기
Push	GitHub로 보내기

비슷한 문구	초보용 뜻
Pull Request / Merge	제안 / 합치기
Permission / Authentication	권한 / 로그인·인증
Conflict	같은 줄을 서로 다르게 고침 (심화)

버튼 전체를 해석하지 말고, **표의 단어만** 찾으시면 됩니다.

Q18. 이건 이 책에서 안 다루나요?

A. 아래는 **의도적으로 뺐**. “다음에 찾아볼 방향”만 적습니다.

주제	이 책	다음에 볼 방향 (힌트만)
GitHub Actions	✗	공식 Docs “Actions” — 9장에서 “나중”으로 둬
리베이스·체리 픽·force push	✗	심화 Git — 지금은 Push·PR만
충돌(conflict) 해결	✗	같은 파일 동시 수정 줄이기 → 공식 “merge conflict” 입문
SSH 키·토큰 심화	✗	초보는 HTTPS + Desktop 로그인 유지
타인·회사 repo PR / Fork	✗	공개 기여는 기본기 후 — 7장은 내 repo
모노레포·대규모 권한	✗	팀·회사 온보딩 자료
VS Code Git 심화	✗	Desktop으로 한 바퀴 먼저

불법·무단 접근·계정 공유 강요는 **어떤 장에도 없음**.

[화면: (선택) 흔한 영문 메시지 예시 카드 — 개인정보·토큰 없이]

복구 체크리스트 (막히면 이 순서)

상황별 **최소** 점검입니다. 위에서 아래로만 가세요.

로그인·권한이 의심될 때

- ☐ https://github.com 인가
- ☐ 브라우저 **Username**이 내 것인가
- ☐ Desktop **같은 계정**인가
- ☐ Private이면 **로그인된 창**으로 여는가
- ☐ 비밀번호는 **본인만** 재설정했는가

“어디에 올렸지?”일 때

- ☐ 커밋만 하고 **Push**를 안 했는가
- ☐ 브라우저 **새로고침** 했는가
- ☐ URL의 Username/저장소이름이 맞는가
- ☐ Desktop **Local path**와 웹 저장소가 짝이 맞는가
- ☐ README(또는 연습 파일) 내용이 **같은가**

클론·커밋·푸시·PR이 안 될 때

- ☐ 클론: Code → HTTPS → path 확인 (Q8~Q10)
- ☐ 커밋: 체크 + 메시지 (Q11)
- ☐ 푸시: 로그인·권한·인터넷 (Q12)
- ☐ PR: 브랜치 → Push → Compare (Q14~Q15)
- ☐ 충돌·Actions·force push면 → **범위 밖 표 (Q18)**

나두 영상 보고 오셨다면 — 끝 점검

“계정 같은지 → Push 했는지 → 웹 새로고침 → URL·폴더 짝”
네 가지만 지나도 대부분 풀립니다.

8장 체크리스트

- ☐ 로그인·인증·권한 막힘에 **먼저 볼 곳**을 말할 수 있다
- ☐ “어디에 올렸지?”를 **커밋→푸시→웹→URL·path** 순으로 되짚는다
- ☐ 클론/푸시/PR 영문 문구를 **표 단어**로 덜 무섭게 읽는다
- ☐ Actions·충돌·force push는 **책 밖**임을 안다
- ☐ 복귀 체크리스트를 **한 번** 눈으로 따라가 봤다

모두 체크되면 9장으로 갑니다.

여기서 한 줄만 더

Q. 책을 다시 몇 장부터 보면 되나요?

A. 증상만 보고 고르면 됩니다.

증상	다시 볼 장
가입·로그인	2장
저장소·README·Public/Private	3장
폴더로 가져오기	4장
기록만 / 아직 웹 없음	5장
웹에 반영	6장
브랜치·PR·Merge	7장
막힘 총정리	8장(지금)
미니 미션·다음 주제	9장

8장을 마치며

1. 막히면 **새 기능보다 점검 순서**
2. 계정은 **브라우저 = Desktop**
3. 안 보이면 **커밋 vs 푸시**부터
4. 영문은 **비슷한 단어**만
5. Actions·충돌·force push는 **다음에**
6. 불법·타인 계정 우회는 **안 함**

다음 **9장**에서는 혼자 반복할 **미니 미션**과 **한 바퀴 복습 체크리스트**, 그리고 “나중에 보면 좋은 주제”만 가볍게 안내합니다.
Actions·심화 Git은 일부러 뒤로 미룹니다.

[화면: 9장 예고 — 미니 미션 3~5개 · 한 바퀴 체크리스트]

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

9장. 다음에 연습할 것

본편 한 바퀴를 끝냈습니다.

이 장은 새 기능을 깊게 배우지 않습니다.

혼자 반복할 미니 미션,

한 바퀴 더 체크리스트,

그리고 “나중에 보면 좋은 주제”만 가볍게 짚습니다.

나두 영상 보고 오셨다면

영상에서 본 도구·팁을 내 저장소에 한 줄이라도 남기는 연습이 다음 단계입니다.

주제별 메뉴는 이 장 끝 부록 두 개로 이어집니다.

화면 문구는 가끔 바뀝니다.

글자가 조금 달라도 비슷한 의미면 같은 단계입니다.

익명 원칙 — 실명·얼굴·실제 이메일·실제 API 키는 README·스크린샷에 넣지 마세요.

연습 저장소 이름은 practice-loop-01처럼 평범하게.

학습 목표

- 혼자 돌릴 미니 미션 3~5개를 고른다
- 계정 → 저장소 → 클론 → 커밋 → 푸시 → PR 체크리스트로 복습한다
- 브랜치 이름·Issues는 이름만 안다 (깊이 없음)
- Actions·심화 Git은 일부러 나중에 둔다
- 부록 두 개를 다음 실습 메뉴로 안다

1. 미니 미션 (혼자 3~5개)

아래 중 오늘 할 수 있는 것만 고르세요.

전부 할 필요 없습니다. 3개만 해도 충분합니다.

미션 A — README 한 줄 더

1. 본인 연습 저장소를 github.com에서 연다
2. README에 날짜 없는 한 줄만 추가한다 (예: 연습: 한 줄 더)
3. 커밋 → 푸시 (5~6장)
4. 웹에서 새로고침해 확인한다

완료: 웹 README와 Desktop(또는 로컬) 내용이 같다.

미션 B — 새 연습 저장소 하나

1. **New repository**로 practice-loop-01(비슷한 이름) 만들기 (3장)
2. **Private + Add a README ON** 권장
3. 클론 → README 한 줄 수정 → 커밋 → 푸시
4. 실명·키가 안 들어갔는지 한 번만 훑는다

완료: 새 repo URL을 북마크(또는 메모)에 적었다.

미션 C — 브랜치 → PR → Merge 한 번 더

1. **본인 저장소**에서 짧은 브랜치 하나 (예: note-readme)
2. 그 위에서 파일 한 줄 고치고 **커밋·푸시**
3. **Compare & pull request** → 제목만 짧게
4. **본인이 Merge** (7장 경로)

완료: 기본 브랜치에 합쳐진 커밋이 웹에 보인다.

타인 저장소 PR은 하지 않습니다.

미션 D — “어디에 올렸지?” 되짚기 (8장 복습)

1. Desktop(또는 로컬)에서 **방금 커밋한 메시지**를 확인한다
2. **Push** 했는지 확인한다
3. 브라우저에서 **같은 계정·같은 저장소 URL**을 연다
4. 안 보이면 8장 복구 **체크리스트**만 본다

완료: “커밋만 / 푸시까지 / 웹 확인”을 말로 구분했다.

미션 E — 메모 파일만 남기기 (선택)

1. 연습 repo에 NOTES.md(또는 memo.md)를 만든다
2. 오늘 막힌 점 **한 줄만** 적는다 (키·비밀번호 금지)
3. 커밋 → 푸시

완료: 웹에서 NOTES.md가 보인다.

[화면: 미니 미션 A~E 체크 칸 — 오늘 고른 3개만 표시]

여기서 막히면

증상별 장은 8장 끝 표(2~7장)로 돌아가세요.
새 버튼보다 **먼저 볼 곳**이 우선입니다.

2. “한 바퀴 더” 체크리스트

소리 내어(또는 손으로) 한 번만 체크합니다.

- ☐ **계정** — github.com에 **본인**으로 로그인된다 (2장)
- ☐ **저장소** — 연습용 repo가 하나 있다 (Public/Private 구분 가능) (3장)

- **클론** — 그 repo를 내쪽 폴더로 가져왔다. 경로를 안다 (4장)
- **커밋** — 파일 고친 뒤 **로컬 기록**을 남겼다. 커밋 ≠ 푸시 (5장)
- **푸시** — GitHub 웹에서 같은 내용이 보인다 (6장)
- **PR** — 본인 repo에서 브랜치 → PR → Merge를 **한 번** 해 봤다 (7장)
- **막힘** — 로그인·어디에 올렸는지·영문 문구는 8장 순서로 본다

한 줄로 다시:

계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

이 여섯 칸이 보이면, 본편 목표는 달성한 것입니다.

[화면: 한 바퀴 다이어그램 — 여섯 칸 체크 완료]

3. 나중에 보면 좋은 것 (이름만)

지금은 **따라 하지 않아도** 됩니다.

이름만 알아 두면, 나중에 검색할 때 덜 낯섭니다.

주제	초보가 알면 좋은 한 줄	이 책에서?
브랜치 이름	fix-..., note-...처럼 짧고 역할이 보이게	7장에서 최소만. 규칙은 나중에
Issues	저장소 안의 할 일·버그 메모 게시판	이름만. 깊게 안 함
Fork	남의 repo를 내 계정 복사본 으로 가져오기	본편 필수 아님. 부록에서 선택
Actions	푸시 후 자동으로 돌아가는 검사·배포 같은 것	일부러 나중에
심화 Git	rebase, cherry-pick, force push 등	일부러 나중에

Issues — 30초만

github.com 저장소 화면 위쪽에 **Issues** 탭이 보일 수 있습니다.

“이 프로젝트의 할 일·질문 목록” 정도로만 기억하세요.

열기·라벨·닫기 절차는 **이 장에서 연습하지 않습니다.**

부록 Lab에서 “에러를 Issue로 적기”가 나오면, 그때 **한 줄 메모** 수준만 따라 가면 됩니다.

브랜치 이름 — 30초만

7장에서 note-readme처럼 **짧은 이름**이면 충분했습니다.

팀마다 규칙이 다릅니다. 지금은 **본인만 알아보기**에 쓰면 됩니다.

4. 일부러 나중에 두는 것

나중에	이유 (한 줄)
GitHub Actions	자동화는 강력하지만, 한 바퀴(계정~PR)가 먼저
rebase / cherry-pick / force push	히스토리를 다시 쓰는 심화·위험 구간
SSH 키 심화·토큰 관리 심화	HTTPS·Desktop 경로로 본편은 충분
타인 repo에 PR·대규모 모노레포	초보 연습 범위 밖

막혀서 Actions 빨간 문구가 보여도,
8장 범위 밖 표처럼 “다음에 찾아볼 주제”로만 두세요.
이 책의 성공 기준은 Actions가 아닙니다.

5. 다음 실습 메뉴 — 부록 두 개

본편을 닫은 뒤, **무엇을 더 연습할지**는 아래 두 부록을 메뉴처럼 고르면 됩니다.
(본문은 여기 다시 쓰지 않습니다. 해당 파일로 이동하세요.)

부록 A — 나두 인기 영상 실습

- 파일: 부록-나두인기영상실습.md
- 역할: 나두Ai(@nadooai) 인기 영상 주제와 깃허브 한 바퀴를 연결
- Lab 1~8 — 영상에서 본 도구·팁을 내 저장소에 작은 흔적으로 남기기
- 전제: 본편 1~5장(가능하면 6~7장)을 한 번 본 뒤

부록 B — 깃허브 트렌드 실습

- 파일: 부록-깃허브트렌드실습.md
- 역할: GitHub Trending 후보로 같은 한 바퀴를 한 번 더
- Lab 1~8 — README·clone(또는 fork)·메모·커밋·푸시 (별 숫자는 적지 않음)
- 전제: 본편 + 가능하면 부록 A Lab 1~2

고르는 법 (30초)

1. 나두 영상을 보고 왔으면 → **부록 A**에서 Lab 하나
2. “요즘 뜨는 repo로 연습”이면 → **부록 B**에서 Lab 하나
3. 둘 다 하고 싶으면 → A 1개 후 B 1개 (하루에 하나만도 OK)

시크릿·실명 규칙은 부록 안내를 그대로 따르세요.

[화면: 부록 A / 부록 B 표지 자리 — Lab 목차만 보이게]

9장 체크리스트

- ☐ 미니 미션을 **3개 이상** 골랐다 (또는 오늘 할 1개를 정했다)
- ☐ **한 바퀴 더** 여섯 칸을 눈으로 체크했다
- ☐ Issues·브랜치 이름은 **이름만** 안다
- ☐ Actions·심화 Git은 **나중에** 두기로 했다
- ☐ 부록 A·B를 **다음 메뉴**로 위치를 안다

모두 체크되면 본편 완주입니다.

이어서는 부록에서 **실습만** 이어가면 됩니다.

나두 영상 보고 오셨다면 — 끝 점검

“한 바퀴 체크 → 미션 하나 → 부록 Lab 하나”

이 순서면 영상과 책이 연결됩니다.

9장을 마치며 (본편 마무리)

1. 새 기능보다 **혼자 반복**
2. 미션은 **3개면 충분**
3. 복습은 **계정→저장소→클론→커밋→푸시→PR**
4. Issues·브랜치 이름은 **힌트만**
5. Actions·심화 Git은 **고의로 미룸**
6. 다음은 **부록 두 개** 실습 메뉴

이 책의 목표였던 한 바퀴는 여기까지입니다.

막히면 8장, 다시 연습하면 이 장 미션,
주제별 확장은 부록으로 이어 주세요.

고맙습니다.

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』 본편을 끝까지 따라와 주셔서.

[화면: 본편 완주 — 한 바퀴 다이어그램 + 부록 A/B 화살표]

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

부록 A. 나두 인기 영상 — 따라 하기 실습

순수 설치·실행·클릭 튜토리얼

나두Ai(@nadooai) 인기 영상에서 본 툴·사이트를 열고 → 쓰고 → (선택) 깃허브에 짧게 저장합니다.

본편 1~9장 = 계정·저장소·클론·커밋·푸시 참고서. 이 부록 = 영상 툴 실습장.

무료 이북입니다. 유료 판매·가격 안내 없음.

브랜드: 나두Ai · @nadooai

UI 문구가 바뀌어도 비슷한 버튼이면 됩니다. 도구 주소는 **본인이 시청한 영상·공식 안내만** 사용하세요. 책에 없는 URL을 지어 내지 마세요. 본문 표기는 항상 영상에서 본 툴 이름.

쓰는 법 (4단계)

1. 아래 Lab 하나를 고른다.
2. 해당 나두 영상을 틀고, 나온 것 중 툴·사이트 1개만 고른다.
3. Lab의 따라하기 번호대로: 연다 → 입력/클릭 → 결과 확인.
4. (선택) 끝 절에서 깃허브에 메모 파일만 3~5클릭으로 남긴다.

공통 저장소 이름 예: my-ai-lab · Lab마다 lab01 ... lab08 폴더.

이미 Lab별 저장소(lab01-free-ai 등)로 나뉘도 됩니다.

시크릿 — 인트로에서 한 번만

해도 됨	하지 말 것
API_KEY= 자리표시	sk-실제키... 커밋
.env.example에 키 이름만	.env 통째 푸시
.gitignore에 .env	스크린샷에 키 전체
실수 시 즉시 키 폐기	“나중에 지울게” 방치

개인 실명·직장 메일·얼굴은 README·캡처에 넣지 마세요.

결과 저장 — 공통 클릭 (Lab 끝에서 재사용)

1. <https://github.com> 로그인.
2. 오른쪽 위 + → **New repository**. 이미 my-ai-lab이 있으면 그 저장소로 이동.
3. 새 저장소면: Repository name my-ai-lab · **Private** · **Add a README file** 체크 ON · **Create repository**.
4. **Add file** → **Create new file** → 파일 이름 칸에 예: lab01/NOTE.md.

- 본문에 Lab에서 적은 메모를 붙여 넣고, 아래 **Commit changes** → 메시지 예 lab01: note → 확인.
- GitHub Desktop을 쓰면: 로컬에서 파일 저장 → **Changes** → 메시지 입력 → **Commit** → **Push origin** (본편 5~6장).

웹에서 만들면 커밋과 동시에 github.com에 반영되는 경우가 많습니다.

실습 목차 (조회수 = 제공값만)

Lab	영상(조회수)	따라 할 일	저장 파일 예
1	이 AI들, 공짜로 씁니다 (~9434)	무료 AI 1개 · 프롬프트 1회	lab01/NOTE.md
2	영상 제작비 0원 만드는 법 (~9175)	영상/음성/자막 툴 · 초안 1개	lab02/NOTE.md
3	이 사이트들, 즐겨찾기 필수입니다 (~5676)	사이트 1개 · 오늘 일 1칸	lab03/NOTE.md
4	매달 나가던 구독료, 끊으세요 (~5076)	무료·OSS 대안 1개 · 실행/데모	lab04/NOTE.md
5	도커 하나로 홈서버 완성 (~2576)	Compose/실행 · 로컬 1회	lab05/NOTE.md
6	요즘 다 이걸로 씁니다 (~1808)	추천 툴·방식 · 채팅/코드 1회	lab06/NOTE.md
7	PDF·사진·전송, 이제 안 사도 됩니다 (~1074)	PDF·이미지·전송 중 1회	lab07/NOTE.md
8	이 사이트들, 있는지도 몰랐죠? (~563)	숨은 툴 · AI 보조 1회	lab08/NOTE.md

공통 패턴 (툴 종류별)

- **웹 AI 채팅**: 주소 열기 → 로그인 → 입력창 → 전송 → 결과 복사.
- **다운로드 앱**: 공식 다운로드 → 설치 → 실행 → New/가져오기 → 저장.
- **사이트·웹툰**: 주소 열기 → 검색/업로드/변환 버튼 → 결과 다운로드·복사.
- **깃허브 OSS**: README Quick start → (선택) Fork/Clone → 첫 명령만.

NOTE.md에 넣을 최소 칸 (복붙용)

Lab:
 영상:
 툴: 영상에서 본 툴 이름
 링크: (확인한 공식 주소만)
 한 일(클릭·명령 요약):
 결과(요약·로컬 경로):
 시크릿 포함 여부: 없음

Lab 1. 「이 AI들, 공짜로 씁니다」 (~9434)

목표: 영상 속 무료(또는 무료 한도) AI **1개**에 내 문장을 넣어 결과 **1회** 받기.

준비

- 브라우저 · (저장 시) github.com 로그인
- 나두 영상 「이 AI들, 공짜로 씁니다」 시청
- 고를 이름: 영상에서 본 톨 이름 **1개**
- 넣을 **내 문장 1개** (회의 메모·메일 초안·코드·숏폼 혹 등 · 키·비밀번호·타인 민감정보 제외)

따라하기

1. 영상을 일시정지하고, 오늘 쓸 도구 이름을 적는다: 영상에서 본 톨 이름.
2. 영상·공식에 나온 **주소**로 브라우저에서 연다. (검색어만 보고 임의 광고 링크를 누르지 말 것)
3. 첫 화면에서 **Log in / Sign up / 시작하기 / Get started** 등 비슷한 버튼이 있으면 본인 계정으로만 진행.
4. 채팅창·프롬프트 칸·Ask/Message 입력란을 찾는다.
5. 아래 톨을 붙여 넣고, 원문만 내 것으로 바꾼다.

역할: (예: 3줄 요약 도우미 / 존댓말 메일 초안 / 코드 원인 후보)

원문:

(여기에 내 글·메모·코드 일부)

부탁: (한 줄로 구체적으로)

제약: 한국어, 모르는 것은 추측이라고 표시

6. **Send / Submit / Generate / 전송**을 누른다.
7. 나온 답 전체를 선택 → 복사(Ctrl+C / Cmd+C) → 메모장·노트앱에 붙여 넣기.
8. 같은 메모 상단에 세 줄을 추가한다.
 - 톨: 영상에서 본 톨 이름
 - 링크: (내가 연 주소)
 - 결과요약: 아래 답을 3~5줄로 줄이거나, 공개 저장소면 요약만 남긴다.
9. 프롬프트에 남은 전화번호·메일·키가 있으면 ***로 바꾼다.
10. (선택) 같은 톨에서 문장만 바꿔 **한 번 더** 전송해도 된다. Lab 기록은 1회분면 충분.

상황별 부탁 한 줄 예 (원문만 교체):

- 아래 메모를 고객 메일 초안, 존댓말 8문장 이내.
- 아래 에러 메시지 기준 원인 후보 2개와 확인할 파일 위치.
- 아래 불릿을 결정사항 / 할 일 / 미정 으로만 분류.
- 얼굴·실명 없이 20초 숏폼 혹 문장 5개.

팁: 결제·업그레이드 창이 뜨면 **닫고**, 영상에서 말한 무료 한도·무료 모드만 다시 찾는다. 이 책은 결제를 요구하지 않는다.

끝 (저장, 선택 · 최대 5클릭)

1. 공통 저장 절차로 lab01/NOTE.md 생성.
 2. 메모장의 툴·링크·프롬프트·결과요약을 붙여 넣기.
 3. Commit message: lab01: free AI once → Commit.
 4. (선택) 저장소에 .gitignore 파일을 만들고 내용에 .env 와 *.key 두 줄.
-

Lab 2. 「영상 제작비 0원 만드는 법」 (~9175)

목표: 무료 영상·음성·자막 툴 1개로 15~30초용 초안 1개까지 만들기.

준비

- 영상 「영상 제작비 0원 만드는 법」 시청
- 주제 한 줄 (예: 커밋과 푸시 차이 20초) · 목표 길이
- 영상에서 본 툴 이름 — 역할: 영상 / 음성 / 자막 중 **하나**
- (저장 시) github.com · 이어폰(선택)

따라하기

1. 오늘 칸을 하나만 고른다: 대본만 / TTS 음성만 / 자막만 / 짧은 편집만.
2. 그에 맞는 영상에서 본 툴 이름을 적고 공식 주소·설치 페이지를 연다.
3. **웹툴**이면: 브라우저에서 연다 → **New project / Create / 시작**.
설치형이면: 공식 다운로드 → 설치 마법사 Next → 앱 실행 → **New**.
4. 대본을 먼저 메모장에 쓴다 (다른 AI로 초안 OK).

혹(1문장):

본론(2~3문장):

끝(1문장):

5. 소리 내어 읽고 15~30초에 맞게 문장을 줄인다.
6. 툴에 대본을 넣는다.
 - 영상 에디터: 텍스트/스크립트 트랙 · 캡션 칸에 붙여 넣기
 - 음성(TTS): 대본 붙여 넣기 → **Generate / 음성 만들기** → 파일 저장
 - 자막: 줄 단위로 나누어 타임라인 또는 자막 칸에 입력
7. **미리보기 / Preview / Play** 한 번.
8. 가능하면 **Export / 내보내기 / Render**(또는 프로젝트 저장)까지. 실패해도 다음 단계로.
9. 저장 경로를 적는다. 예: Downloads/lab02-draft · 문서/나두랩/lab02.
10. 메모에 고정: 툴 이름 · 역할 · 스크립트 전문 · 길이·해상도·음성 종류 · 로컬 경로.

팁: 큰 MP4/WAV는 깃허브에 올리지 않는다. 경로와 스크립트만 남긴다. 배경음악은 영상에서 안 내한 무료 소스만, 불확실하면 무음.

끝 (저장, 선택)

1. lab02/NOTE.md에 단계 10 메모를 붙여 넣기.
2. Commit: lab02: draft note.
3. 바이너리 파일은 Add하지 않음.

Lab 3. 「이 사이트들, 즐겨찾기 필수입니다」 (~5676)

목표: 영상 속 사이트 1개로 오늘 할 일 1칸을 실제로 처리하기.

준비

- 영상 시청
- 오늘 할 일 1개 (레퍼런스 찾기 · 색 조합 · 문서/용어 확인 · 아이콘 · 퀴즈 등)
- 영상에서 본 툴 이름(사이트) · 확인한 공식 주소
- (저장 시) github.com

따라하기

1. 할 일과 겹치는 사이트 하나만 고른다. 메모: 사이트: 영상에서 본 툴 이름 / 일: ____.
2. 공식 주소를 주소창에 입력해 연다.
3. 쿠키·동의 배너가 있으면 필수만 처리하고 닫는다.
4. 회원 가입이 필수면 본인 메일로 Sign up. 아니면 게스트·검색만.
5. 할 일 유형별 클릭:
 - **검색형**: 검색창 클릭 → 키워드 입력 → Enter → 결과 제목 3개를 메모에 복사
 - **생성·조합형**: 옵션/팔레트/템플릿 선택 → **Apply / Generate / Download**
 - **업로드형**: **Upload / 파일 선택** → 개인정보 없는 샘플만 → 변환·다운로드
 - **문서형**: 메뉴에서 해당 페이지 진입 → 필요한 문단·표만 복사
6. 결과물 위치 확인: 다운로드 폴더 파일명, 또는 복사한 텍스트가 메모에 있는지.
7. (선택) AI 채팅을 열고 결과를 붙여 넣은 뒤 아래를 표로 정리해 줘. 기준: ___ 1회 전송.

팁: 로그인·지역 제한으로 막히면 영상 속 **다른 사이트 1개**로 바꿔 1~6만 반복한다.

끝 (저장, 선택)

1. lab03/NOTE.md 작성 예:

사이트: 영상에서 본 툴 이름
링크: (확인 주소)
오늘 일:
한 클릭/입력:
결과(요약·경로):

2. Commit: lab03: site once.
-

Lab 4. 「매달 나가던 구독료, 끊으세요」 (~5076)

목표: 유료 대신 쓸 무료·OSS 대안 **1개**를 열어, 같은 종류 입력으로 **1회** 실행하기 (웹 데모 또는 로컬).

준비

- 영상 시청 · 대체할 루틴 한 줄 (예: 유료 AI로 하던 요약)
- 영상에서 본 툴 이름 또는 공개 깃허브 프로젝트 주소
- github.com 로그인 · (선택) GitHub Desktop · 터미널
- 연습용 입력(더미 문단·공개 샘플 · 실명·카드·사내 로그 금지)

따라하기

1. 대안 이름·주소를 메모에 적는다. 출처 = 영상·공식·깃허브만.
2. 브라우저에서 그 페이지를 연다.
3. 깃허브 저장소면: README에서 **Getting started / Quick start / Install** 절만 스크롤해 연다.
4. (선택) 저장소 상단 **Fork** → Create fork. 내 계정에 복사본이 생긴다.
5. 아래 **A** 또는 **B** 중 하나만 한다.

A. 웹 데모·호스팅 앱

1. README 또는 사이트 mid의 **Demo / Live / Try / App** 링크를 연다.
2. 입력란에 연습용 문장·파일을 넣는다.
3. **Run / Submit / Generate** 클릭.
4. 결과 화면을 보고 메모: 입력종류 / 결과한줄 / 에러문(있으면, 키·이메일 가림).

B. 클론 후 로컬

1. **Code** 버튼 → **HTTPS** → URL 복사.
2. Desktop: **File** → **Clone repository** → URL 붙여 넣기 → Local path 선택 → Clone.
터미널: git clone 뒤에 URL을 붙여 실행 (본편 4장).
3. 클론된 폴더에서 README의 **첫 설치 명령**만 복사해 실행한다. (예: npm i · pip install -r requirements.txt · 문서에 있는 명령만)
4. 키가 필요하면 로컬에 .env를 만들고 실값을 넣는다. 저장소에는 아래만 둔다.

```
# .env.example
API_KEY=
```

5. .gitignore에 .env가 있는지 확인하고 없으면 한 줄 추가.

- 터미널에서 폴더 진입 후: grep -n "\.env" .gitignore 로 확인해도 된다.
- 웹만 쓰면 github.com 파일 목록에서 .gitignore를 열어 .env 줄을 추가·커밋.

6. README의 실행 명령으로 기동 → 브라우저 또는 CLI에서 연습 입력 1회.

7. git status에 .env가 **안** 뜨는지 확인한다. 뜨면 Add하지 말고 ignore부터 수정.

8. 공통 메모 작성: 이전: ___ / 대안: 영상에서 본 툴 이름 / 경로: A또는B / 명령·클릭: ___ / 결과또는에러: ___.

팁: 설치가 깨져도 가린 에러+시도 명령을 NOTE.md에 남기면 된다. 카드 해지·환불 절차는 다루지 않는다.

끝 (저장, 선택)

1. lab04/NOTE.md (+ 있으면 .env.example만).

2. (선택) 본인 저장소 **Issues** → **New issue** → 제목 lab04: blocked → 본문에 가린 에러·다음에 볼 README 절 제목.

3. Commit: lab04: alt trial.

Lab 5. 「도커 하나로 홈서버 완성」 (~2576)

목표: 영상 속 홈서버·셀프호스트 예시 1개의 Docker/Compose 안내를 따라 **localhost**로 열거나, 같은 문서로 실행 메모를 완성하기.

준비

- 영상 시청 · 목적 한 줄 (로컬 노트 / 파일함 / 봇 등)
- 영상에서 본 툴 이름 · 공식 깃허브/문서 URL
- PC 여유 공간 · Docker 설치 여부 확인
- 연습용 빈 데이터 폴더 (중요 원본 금지) · github.com(저장 시)

따라하기

1. 후보를 적는다: 영상에서 본 툴 이름.
2. 공식 README·문서에서 Docker / Docker Compose / Quick start 제목을 찾아 연다.
3. 문서의 예시 `compose.yaml` / `docker-compose.yml` 코드 블록을 복사할 준비를 한다. **예시에 있는 비밀번호·토큰은 연습용으로 바꾸고, 그 실값을 깃에 넣지 않는다.**
 - ports 예: "8080:80" 이면 브라우저는 `http://localhost:8080`.
 - volumes 경로는 단계 4에서 만든 빈 폴더만 연결.
4. Docker가 설치되어 있으면:
 1. 빈 작업 폴더 생성. 예: `mkdir ~/nadoo-lab05 && cd ~/nadoo-lab05`
 2. 데이터 전용 빈 폴더. 예: `mkdir ~/nadoo-lab05-data`
 3. 문서 안내에 따라 `compose` 파일을 저장.
 4. 문서에 적힌 기동 명령만 실행 (예: `docker compose up -d` — **문서에 있는 철자 그대로**).
 5. 출력에 포트 번호가 보이면 브라우저에서 `http://localhost:포트` 접속.
 6. UI가 열리면 **테스트 메모·테스트 파일 1개만** 추가.
 7. 문서의 종료 절을 찾아 `docker compose down` 등 **적힌 명령**을 메모에 복사.
5. Docker가 없으면: 단계 2의 명령·이미지 이름·포트·볼륨 경로를 NOTE.md에 그대로 옮겨 적고, Docker 설치 후 재시도 한 줄을 남긴다. 무리한 설치 강제 없음.
6. 캡처 시 비밀번호·토큰·집 주소를 자른다.
7. 공유기 포트포워딩·HTTPS·외부 공개 IP는 **설정하지 않는다**. 이 Lab은 로컬만.

팁: `port is already allocated` 류 메시지면 문서의 ports 매핑을 다른 번호로 바꾸는 방법을 찾아 수정하거나, 컨테이너를 내리고 메모만 커밋한다.

끝 (저장, 선택)

1. lab05/NOTE.md에 후보·사용한 명령(실 시크릿 없음)·포트·테스트 여부·종료 명령.
 2. Commit: lab05: compose notes.
-

Lab 6. 「요즘 다 이걸로 씁니다」 (~1808)

목표: 영상에서 “요즘 쓴다”고 한 툴·모델·방식 **1개**를 실제 채팅 또는 코드 화면에 **1회** 적용하기.

준비

- 영상 시청 · 영상에서 본 툴 이름
- 평소 쓰는 AI 채팅 탭 또는 코드 편집기
- 시험용 작은 작업 1개 (함수 설명 · 톤 변경 · 할 일→표 등)
- (저장 시) github.com

따라하기

1. 후보와 진입점을 적는다. 예: 툴/방식: 영상에서 본 툴 이름 / 진입: 사이트|앱|모델선택|플러그인|프롬프트템플릿.
2. 영상에서 보여 준 화면까지 이동한다.
 - **웹:** 주소 열기 → 로그인 → 모델/모드 드롭다운에서 영상과 같은 항목 선택
 - **앱:** 설치·실행 → 설정에서 해당 옵션 ON
 - **플러그인/확장:** 마켓플레이스에서 이름 검색 → Install → 에디터/채팅에서 활성화
 - **템플릿:** 저장된 프롬프트·스니펫 불러오기
3. 평소와 **같은 작은 작업문**을 입력창에 넣는다.
4. **Send / Run / 적용** 1회.
5. 결과를 복사해 메모에 붙인다.
6. 메모 하단 두 줄:

전: (평소 방식 한 줄)

후: (이번 툴·방식 한 줄)

7. 링크·모델명·옵션명은 화면에 보이는 그대로 적는다 (기억나는 별명만 쓰지 말 것).

팁: 유료 플랜만 열려 있으면 무료 한도·무료 모델만 사용. 비교가 필요하다면 동일 질문을 옛 방식 1번·새 방식 1번만.

끝 (저장, 선택)

1. lab06/NOTE.md — 고른 것 · 링크 · 전 · 후 · 결과 요약.
 2. Commit: lab06: try once.
-

Lab 7. 「PDF·사진·전송, 이제 안 사도 됩니다」 (~1074)

목표: 무료·OSS 1개로 PDF·이미지·파일 전송 중 하나를 더미 파일로 1회 처리하기.

준비

- 영상 시청 · 종류 1택: PDF / 사진 / 전송
- 영상에서 본 툴 이름 · 공식 주소 또는 설치 페이지
- **더미 파일** 1개 (주민번호·계약서·신분증 스캔 금지)
- (저장 시) github.com

따라하기

1. 메모: 종류: PDF|사진|전송 / 툴: 영상에서 본 툴 이름.
2. 웹이면 주소 오픈 · 설치형이면 공식 설치 후 실행 · OSS면 README의 실행 절 오픈.
3. UI에서 **파일 선택 / Open / Upload / 열기** 버튼을 찾는다.
4. 더미 파일이 없으면 지금 만든다.
 - PDF: 메모장에 lab07 dummy 몇 줄 → 인쇄 대화상자에서 PDF 저장, 또는 무해한 기존 PDF
 - 사진: 풍경·물건 사진 (사람 얼굴·신분증 제외)
 - 전송: 위 파일 중 하나 · 가능하면 zip
5. 파일을 툴에 넣는다.
6. 영상에서 강조한 기능 **하나만** 실행한다. 예: 합치기 · 분할 · 압축 · 리사이즈 · 포맷 변환 · 임시 링크 생성.
7. **Download / Save / 복사 링크**로 결과를 받고 경로·URL을 메모.
8. 전송형이면: 만료 시간·비밀번호 옵션을 확인하고, 테스트 후 **만료·삭제**가 있으면 즉시 켜다.
9. 다운로드 폴더·클라우드 최근 항목에 민감 파일이 섞이지 않았는지 파일명만 훑는다.
10. 메모 고정: 툴 · 클릭 순서(3~6줄) · 결과 로컬 경로 · 더미만 사용: 예.

팁: 웹 업로드가 불안하면 로컬 설치형·오프라인 변환만. 경로 오류 시 파일을 Desktop/lab07-dummy.pdf로 옮겨 다시 선택.

끝 (저장, 선택)

1. lab07/NOTE.md에 단계 10만. **결과 PDF·이미지·zip은 깃에 추가하지 않음.**
 2. Commit: lab07: file once.
-

Lab 8. 「이 사이트들, 있는지도 몰랐죠?」 (~563)

목표: 잘 안 알려진 툴 1개로 AI 작업(또는 할 일)용 **재료 1개**를 만들고, AI에 1회 넣기.

준비

- 영상 시청 · 작은 작업 1개 (설명문 · 표 · 색·단위 변환 · 자료 3개 요약 등)
- 영상에서 본 툴 이름 · 확인한 공식 링크
- AI 채팅 탭(또는 메모장) · (저장 시) github.com

따라하기

1. “처음 알았거나 잊고 있던” 항목 1개를 고른다.
2. 공식 링크로 사이트를 연다. 비회원 사용 가능하면 가입 없이 진행.
3. 첫 화면에서 핵심 입력란·버튼을 찾는다.
4. 재료 유형별:
 - **검색:** 키워드 1개 → 결과 제목·한 줄 요지 3개 메모
 - **변환:** 값·파일·색코드 등 입력 → **Convert / 변환** → 결과 복사
 - **정리:** 템플릿·표·체크리스트 생성 → 내 항목 3칸만 채움
5. AI 채팅을 옆에 두고, 재료를 붙여 넣는다.
6. 지시 1회 전송. 예:

아래 재료를 바탕으로 초보용 설명 5문장을 써 줘.

재료:

(붙여 넣기)

7. AI 답을 메모 하단에 요약 3줄로 남긴다.
8. 카탈로그용 한 줄을 확정한다: 정확한 이름 · 한 줄 용도 · 링크 · 오늘사용: YES.

팁: 사이트 접속 불가·지역 제한이면 영상 속 **다른 1개**로 1~8만 반복.

끝 (저장, 선택)

1. lab08/NOTE.md 예:

이름	용도	링크	사용
-----	-----	-----	-----
영상에서 본 툴 이름	(한 줄)	(확인 주소)	YES

적용:

- 작업:
- 툴에서:

- AI에:
- 결과요약:

2. Commit: lab08: hidden tool.

마치며

- 커밋·푸시가 아직 어색하면 본편 **5~6장**을 한 바퀴.
- 다음 메뉴: **부록 B** 『깃허브 트렌드 → 내 AI에 끌어오기』.
- 조회수 숫자는 매니저 제공값. 플랫폼 제목이 조금 달라도 **같은 주제 Lab**을 고르면 됩니다.
- 도구 공식 주소는 항상 **시청한 영상·공식 안내**에서만. 가짜 제품 URL 금지.

나두Ai · 부록 A · 따라 하기 · 무료 이북

부록 B. 깃허브 공개 저장소 설치 따라 하기

예시 레포(교체 가능). Trending 날짜에 묶이지 않음 — AI 초보가 설치해 보기 좋은 공개 저장소 예시.

브랜드: 나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

방식: 따라 하기식 가벼운 설치·실행 튜토리얼

배포: 무료 전자책 · 익명 연습 권장

전제: 본편 1~5장(가능하면 6장)을 한 번 본 뒤 오면 더 쉽습니다.

이 부록은 「유명 레포 구경」이 아닙니다. **설치(또는 다운로드) → 한 번 실행 → 결과 파일을 내 깃허브에 짧게 남기기만** 합니다.

이 부록을 쓰는 법

매 Lab은 같은 흐름입니다.

1. 목표 읽기
2. 공식 URL 열기
3. 번호 스텝대로 설치·실행
4. 결과 파일을 내 저장소에 커밋 3스텝

- 명령은 공식 **README**가 바뀌면 README를 우선합니다. 이 책은 초보용 한 줄 예시입니다.
- API 키·비밀번호·`.env` 실값·개인 문서는 **절대 커밋하지 마세요**.
- Star만 누르고 끝내지 마세요. **설치·실행 결과 파일**이 있으면 충분합니다.

공통: 결과 저장소 만들기 (한 번만 해도 됨)

1. <https://github.com/new> 에서 `nadoo-install-lab` (또는 Lab마다 `nadoo-lab-1` ...) 만들기
2. **Private** 권장 · **Add a README ON** · **Create repository**
3. Lab마다 폴더를 나눠도 됩니다. 예: `lab-01/`, `lab-02/` ...

공통: 결과 커밋 3스텝 (웹에서도 가능)

1. 내 저장소에서 **Add file** → **Create new file** (또는 Desktop에서 파일 추가)
2. Lab에서 정한 결과 파일 이름·내용 넣기 (키·실문서 제외)
3. **Commit changes** → 메시지 예: `lab-N: install note`

Desktop을 쓰면: 파일 저장 → Changes에서 커밋 → Push (본편 5~6장).

Python이 없을 때

Lab 1·5·6·7은 Python이 필요합니다. 없으면 python.org에서 설치한 뒤 터미널을 새로 여세요.
Lab 2(프롬프트)·Lab 4(gh만)는 Python 없이 진행할 수 있습니다.

명령이 README와 다르면

패키지·CLI 문법은 자주 바뀝니다. **빨간 글자**가 나면 이 책 예시를 고집하지 말고, 해당 저장소 README의 Installation / Usage를 복사하세요.

시크릿 (모든 Lab)

올리면 안 됨	올려도 됨
.env 실값, API 키, 토큰	.env.example (키 이름만)
신분증·계약서·회사 문서	연습용 가짜 텍스트·요약
실명·직장 메일 스크린샷	가린 화면 설명 한 줄

Lab별 남길 파일 (짧게)

Lab	올리면 좋은 것	올리면 안 되는 것
1	NOTE.md, 연습용 out.md 일부	실제 기밀 PDF
2	NOTE.md (프롬프트 제목만)	AI 대화 전문·개인정보
3	NOTE.md (설치·허용 소스만)	받은 동영상 파일
4	NOTE.md (gh 버전)	gh 토큰
5	hello_openai.py, .env.example, .gitignore	.env
6	NOTE.md	HF 토큰·대용량 가중치
7	app.py, NOTE.md	불필요

실습 목차 (가벼운 7 Lab)

Lab	저장소	한 줄로 하는 일
1	microsoft/markitdown	pip 설치 → 파일 1개 Markdown 변환
2	f/awesome-chatgpt-prompts	클론/다운로드 → 프롬프트 1개 복사해 AI에 사용
3	yt-dlp/yt-dlp	설치 → 허용된 샘플 URL 1개 받기
4	cli/cli	GitHub CLI 설치 → gh --version
5	openai/openai-python	pip → hello 스크립트 뼈대 (키는 .env)

Lab	저장소	한 줄로 하는 일
6	huggingface/huggingface_hub	pip → CLI로 아주 작은 파일 받기(또는 로그인 문서만)
7	gradio-app/gradio	pip → 5줄 hello UI

이번 목록에서 제외 (Skip)

무거운 설치·계정·Docker·에이전트 하네스 위주는 **따라 하기 A**에서 뺍니다.

항목	처리	이유
paperless-ngx/paperless-ngx	제외	Docker·문서함 기동 부담
n8n-io/n8n	제외	워크플로·의존성 무거움
affaan-m/ECC	제외	에이전트 하네스·설계 중심
BuilderIO/agent-native	제외	에이전트 앱·매핑 숙제형
anthropics/claude-code	제외	계정·설치 선택 부담
Open-Dev-Society/OpenStock	제외	실행/환경이 가벼운 설치 튜토리얼에 덜 맞춤
mihail911/modern-software-dev-assignments	제외	과제·학습 계획형 (설치 실습 아님)
higgsfield-ai/higgsfield 등 GPU ML	제외	초보 설치 범위 밖
xtekky/gpt4free	제외	스케치·약관 리스크

레포가 없거나 README가 크게 바뀌면 **같은 난이도의 다른 공개 저장소**로 바뀌도 됩니다. 흐름(설치 → 실행 1번 → 커밋)만 유지하세요.

Lab 1. Markdown — microsoft/markitdown

목표

연습용 파일 1개를 Markdown으로 바꾸고, 변환 결과(또는 설치 메모)를 내 깃허브에 남긴다.

공식 URL

<https://github.com/microsoft/markitdown>

설치·실행

1. 터미널에서 Python 확인 (3.10+ 권장):

```
python3 --version
```

Windows에서 안 되면 `python --version`.

2. (권장) 가상환경:

```
python -m venv .venv  
source .venv/bin/activate
```

Windows 활성화는 공식 README를 따릅니다.

3. 설치:

```
pip install 'markitdown[all]'
```

4. **연습용** PDF/docx/txt만 준비합니다. 신분증·계약서·회사 기밀 금지.

5. 변환:

```
markitdown 샘플.pdf -o out.md
```

또는:

```
markitdown 샘플.pdf > out.md
```

6. out.md를 열어 글자가 보이면 성공입니다.

7. PDF 의존성 오류가 나면 README의 extras 안내를 보고 `pip install 'markitdown[pdf]'` 등을 시도합니다.

[화면: 터미널 markitdown 실행 · out.md 일부]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-1 (또는 nadoo-install-lab/lab-01/) 열기
 2. NOTE.md에 설치 한 줄 + (연습용이면) out.md 일부 업로드. 민감하면 “로컬에만 둬”이라고만 적기
 3. 커밋: lab-1: markdown convert
-

Lab 2. Awesome ChatGPT Prompts — f/awesome-chatgpt-

prompts

목표

저장소에서 프롬프트 1개를 골라, ChatGPT/Claude 등에 붙여 넣고, 사용한 프롬프트 제목만 내 깃허브에 남긴다. (무거운 설치 없음)

공식 URL

<https://github.com/f/awesome-chatgpt-prompts>

설치·실행

1. 브라우저에서 위 URL을 엽니다.
2. **Code** → **Download ZIP** 또는 HTTPS 클론(본편 4장):

```
git clone https://github.com/f/awesome-chatgpt-prompts.git
```

3. README 또는 목록에서 **프롬프트 1개**만 고릅니다.
4. 평소 쓰는 AI 채팅을 열고, 그 프롬프트를 붙여 넣은 뒤 **짧은 질문 1개**만 해 봅니다.
5. 답변 전문·개인 대화는 올리지 마세요.

[화면: 저장소 README · 프롬프트 하나 선택]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-2 열기
2. NOTE.md에 예:

Lab 2

- 날짜:
- 쓴 프롬프트 제목(또는 한 줄 요약):
- 사용한 AI: (예: ChatGPT)
- 결과 한 줄: (성공/실패만)

3. 커밋: lab-2: one prompt trial
-

Lab 3. yt-dlp — yt-dlp/yt-dlp

목표

yt-dlp를 설치하고, **저작권·이용이 허용된** 짧은 샘플 URL 1개만 받아 본다.

저작권: 본인 영상, 공개 도메인, 또는 다운로드를 **명시 허용**한 자료만. 남의 유료·저작권 콘텐츠를 무단으로 받지 마세요.

연습만 할 때는 직접 찍은 짧은 클립을 로컬에 두고, “허용 URL로 1회 받음”만 NOTE.md에 적어도 됩니다.

공식 URL

<https://github.com/yt-dlp/yt-dlp>

설치·실행

1. 공식 README의 **Installation**을 엽니다. OS별 원라이너가 있으면 그걸 우선합니다.
2. Python이 있으면 예:

```
pip install -U yt-dlp
```

Windows winget 등이 README에 있으면 그대로 따라도 됩니다.

3. 버전 확인:

```
yt-dlp --version
```

4. **허용된** 짧은 URL 하나로 받기 (URL은 본인이 고름):

```
yt-dlp "https://예시-허용-URL"
```

5. 받은 파일이 폴더에 보이면 성공입니다. 큰 파일·불법 소스는 올리지 마세요.

[화면: yt-dlp --version · 다운로드 완료 한 줄]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-3 열기
 2. NOTE.md에: 설치 방법 한 줄 · “허용 소스에서 1회 받음” · **동영상 파일 자체는 커밋하지 않기**
 3. 커밋: lab-3: yt-dlp install note
-

Lab 4. GitHub CLI — cli/cli

목표

gh를 설치하고 gh --version이 나오게 한다. (로그인·클론은 선택)

공식 URL

<https://github.com/cli/cli>

설치·실행

1. README의 **Installation**에서 내 OS 항목을 고릅니다.
2. 예 (환경마다 다름 — README 우선):

- macOS: brew install gh
- Windows: winget install --id GitHub.cli
- Linux: 배포판별 안내 따름

3. 새 터미널을 연 뒤:

```
gh --version
```

버전 숫자가 보이면 Lab 본미션 완료입니다.

4. (선택) 계정 연동:

```
gh auth login
```

화면 안내를 따릅니다. 강제 아님. 부담되면 버전 확인만으로 끝냅니다.

5. (선택) 내 저장소 하나 클론:

```
gh repo clone 내계정/내저장소
```

[화면: gh --version 출력]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-4 열기
 2. NOTE.md에 OS · 설치 방법 · gh 버전 문자열만 적기 (토론 금지)
 3. 커밋: lab-4: gh version
-

Lab 5. OpenAI Python — openai/openai-python

참고: 이 Lab만 **API 키 발급·과금** 개념이 들어갑니다. 키는 .env에만 두고 커밋하지 마세요. 부담되면 Lab 1_{4.6}7을 먼저 해도 됩니다.

목표

패키지를 설치하고, API 키는 .env에만 두는 hello 스크립트 **뼈대**를 만든다. 키는 커밋하지 않는다.

공식 URL

<https://github.com/openai/openai-python>

실제 API 호출은 **본인 계정·요금·약관**을 확인한 뒤에만. 키가 없으면 파일 뼈대 + .env.example만으로도 이 Lab은 끝낼 수 있습니다.

설치·실행

1. 가상환경(권장) 후:

```
pip install openai python-dotenv
```

2. 프로젝트 폴더에 .gitignore를 만들고 한 줄 추가:

```
.env
```

3. .env.example (커밋용 — **가짜 자리만**):

```
OPENAI_API_KEY=CHANGE_ME
```

4. 로컬 전용 .env (커밋 금지):

```
OPENAI_API_KEY=여기에_본인_키
```

5. hello_openai.py 뼈대 예:

```
import os
from dotenv import load_dotenv

load_dotenv()
key = os.getenv("OPENAI_API_KEY")

if not key or key == "CHANGE_ME":
    print("키가 없습니다. .env만 로컬에 준비하고 커밋하지 마세요.")
else:
```

공식 README의 최신 클라이언트 예제를 여기에 붙입니다.

```
print("키 로드됨(값은 출력하지 마세요). README 예제로 한 번만 호출해 보세요.")
```

6. 실행:

```
python hello_openai.py
```

키 문자열을 print하지 마세요. README의 Chat/Responses 예제는 **공식 문서 버전**을 복사합니다.

[화면: .gitignore에 .env · hello 스크립트]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-5 열기
 2. 올릴 것: hello_openai.py · .env.example · .gitignore · NOTE.md 한 줄. **올리지 말 것: .env**
 3. 커밋: lab-5: openai hello skeleton
-

Lab 6. Hugging Face Hub — `huggingface/huggingface_hub`

목표

`huggingface_hub`를 설치하고, CLI로 **아주 작은** 공개 파일을 받거나, 로그인 문서만 읽고 메모를 남긴다.

공식 URL

https://github.com/huggingface/huggingface_hub

설치·실행

1. 설치:

```
pip install -U huggingface_hub
```

2. CLI 확인:

```
hf --help
```

3. 최소 경로 A — 작은 공개 파일 다운로드

README/hf download 예시를 보고, **용량이 작은** 공개 파일 하나만 받습니다.

예 (모델·이름은 README·Hub 화면 기준으로 본인이 고름):

```
hf download 작은공개파일/또는모델 --include "*.txt" --local-dir ./hf-sample
```

명령이 바뀌었으면 공식 README를 그대로 따릅니다.

4. 최소 경로 B — 받기 부담되면

README의 로그인(`hf login`) 설명만 읽고, 오늘은 로그인하지 않아도 됩니다. NOTE.md에 “문서만 확인”이라고 적으면 됩니다.

5. 토큰·캐시 경로에 있는 비밀 파일은 커밋하지 마세요.

[화면: `hf --help` 일부]

내 저장소에 남기기 (3스텝)

1. `nadoo-lab-6` 열기

2. NOTE.md: 설치 확인 · (A) 받은 파일 이름만 / (B) 문서만 확인. **토큰·대용량 가중치 금지**

3. 커밋: `lab-6: huggingface_hub note`

Lab 7. Gradio — gradio-app/gradio

목표

Gradio를 설치하고, 5줄 안팎 hello UI를 로컬에서 한 번 띄운다.

공식 URL

<https://github.com/gradio-app/gradio>

설치·실행

1. Python 3.10+ 확인 후:

```
pip install --upgrade gradio
```

2. app.py 작성:

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
```

(공식 README 예제가 더 최신이면 README를 우선합니다.)

3. 실행:

```
python app.py
```

4. 터미널에 나온 주소(예: `http://127.0.0.1:7860`)를 브라우저로 엽니다.
5. 이름 칸에 글자를 넣고 Submit → 인사가 보이면 성공.
6. 끝낼 때 터미널에서 Ctrl+C. `share=True`는 초보 Lab에서는 생략해도 됩니다.

[화면: 로컬 Gradio 인사 화면(실명 가림)]

내 저장소에 남기기 (3스텝)

1. nadoo-lab-7 열기
 2. app.py + NOTE.md(“로컬에서 launch 성공”) 업로드
 3. 커밋: lab-7: gradio hello
-
-

레포를 바꿀 때 (짧게)

예시 7개는 고정 커리큘럼이 아닙니다. 아래만 지키면 다른 공개 저장소로 바뀌도 됩니다.

1. 초보가 몇 명령 또는 브라우저 몇 클릭으로 끝낼 수 있을 것
2. 공식 README에 Installation / Usage가 있을 것
3. GPU·Docker·유료 계정 강제가 없을 것 (있으면 Skip 표에 넣고 대체)
4. 결과물로 NOTE.md 또는 작은 코드/변환 파일 1개를 내 저장소에 남길 것

교체 후보 예: psf/requests(pip hello), sharkdp/bat(간단 CLI), BurntSushi/ripgrep(간단 CLI).
이 책 본문 예시는 위 7개를 기준으로 둡니다.

부록 B를 끝낸 뒤

1. nadoo-lab-1 ... 7 (또는 한 저장소의 lab-0N/)에 **NOTE·결과 파일**이 보이는지 확인합니다.
2. 7개를 다 할 필요 없습니다. **하나라도** 설치→실행→커밋이면 이 부록 습관은 성공입니다.
3. 레포가 사라지거나 너무 무거워지면, 같은 난이도 공개 저장소로 교체하고 이 부록 흐름만 유지하세요.
4. 부록 A(나두 영상 실습)와 주제가 겹치면, 같은 날 둘 다 할 필요 없습니다.
5. 본편 8장 Q&A · 9장 다음 연습으로 돌아가 “한 바퀴”를 이어 가면 됩니다.

한 줄 복습

공개 저장소는 구경거리가 아니라, **설치해 보고 결과 파일만 내 깃허브에 남기는 연습 재료**입니다.

나두Ai · 부록 B · 가벼운 설치 따라 하기 · 예시 레포 7 · 무료 전자책 · 2026-09-21

고지 · 법적 주의

※ 이 문서는 안내 체크리스트이며 법률 의견이 아닙니다.

『AI 하는 사람을 위한 깃허브 초간단』 (나두Ai)을 이용하기 전에 한 번 읽어 주세요.

1. 저작권 · 이용 범위

- 이 전자책 PDF의 저작권은 **나두Ai**(익명 발행)에 있습니다.
 - 무료로 제공하며, **개인 학습용**으로 이용할 수 있습니다.
 - 무단 **재판매·유료 재배포·타인 명의의 유료 상품화**는 금지합니다.
 - 공식 배포 주소(<https://tinalinkeom.vercel.app/ebook>)를 공유하는 것은 허용합니다.
-

2. 제3자 상표 · 비제휴

- GitHub, TikTok 및 각종 AI·오픈소스 명칭은 각 권리자의 상표일 수 있습니다.
 - 이 전자책·나두Ai는 GitHub 등의 **공식 교육·제휴·인증 커리큘럼이 아닙니다**.
 - 화면 문구는 서비스 업데이트로 바뀔 수 있으니, 비슷한 위치·비슷한 문구를 기준으로 따라가 주세요.
-

3. 제3자 코드 · 저장소

- 실습은 공개 저장소 링크·클론·포크·설치 안내입니다. 이 PDF가 타인의 소스코드를 상품으로 재판매하지 않습니다.
 - 각 저장소의 **라이선스(MIT, Apache 등)**는 독자가 직접 확인·준수해야 합니다.
 - 예시로 든 저장소는 시점에 따라 삭제·변경될 수 있습니다.
-

4. 나두Ai 콘텐츠

- TikTok·YouTube 등 나두Ai 영상은 별도 작품입니다.
 - 이 전자책은 **따라하기 가이드**이며, 영상 파일 자체를 포함·재배포하지 않습니다.
 - 영상은 삭제되거나 노출이 바뀔 수 있습니다.
-

5. 교육 목적 면책

- 학습·실습 안내입니다. 특정 결과·수익·취업·보안 완벽을 보장하지 않습니다.
- 도구·API·요금제·화면은 변경될 수 있습니다.

- API 키·비밀번호·.env 등 비밀값이 유출되지 않도록 관리할 책임은 독자에게 있습니다. 커밋·공유 전에 한 번 더 확인하세요.

6. 금지 안내

불법 스크래핑, 서비스 약관 위반, 무단 침해, 악성 자동화 등에 이 자료를 사용하지 마세요.

7. 제공 방식 · 문의

항목	내용
브랜드	나두Ai
상품	『AI 하는 사람을 위한 깃허브 초간단』 PDF (무료)
받는 곳	https://tinalinkeom.vercel.app/ebook
직접 파일	https://tinalinkeom.vercel.app/guides/nadoo-github-guide.pdf
문의	나두Ai 채널(@nadooai) 및 https://tinalinkeom.vercel.app

현재는 **무료** 제공됩니다. 유료 판매·환불 규정은 적용하지 않습니다.

나두Ai